

CHƯƠNG 18. HỌC TĂNG CƯỜNG

Học tăng cường (RL) là một trong những lĩnh vực thú vị nhất của học máy hiện nay, và cũng là một trong những lĩnh vực lâu đời nhất. Nó đã xuất hiện từ những năm 1950, tạo ra nhiều ứng dụng thú vị trong những năm qua, đặc biệt trong các trò chơi (ví dụ: TD-Gammon, một chương trình chơi Backgammon) và trong điều khiển máy móc, nhưng hiếm khi trở thành tin tức nóng hổi. Tuy nhiên, một cuộc cách mạng đã diễn ra vào năm 2013, khi các nhà nghiên cứu từ một công ty khởi nghiệp của Anh tên là DeepMind đã chứng minh một hệ thống có thể học chơi gần như bất kỳ trò chơi Atari nào từ đầu, cuối cùng vượt qua con người trong hầu hết các trò chơi đó, chỉ sử dụng các pixel thô làm đầu vào và không có bất kỳ kiến thức nào về luật chơi trước đó. Đây là thành tựu đầu tiên trong một loạt các thành tựu đáng kinh ngạc, đỉnh điểm là chiến thắng của hệ thống AlphaGo của họ trước Lee Sedol, một kỳ thủ cờ vây chuyên nghiệp huyền thoại, vào tháng 3 năm 2016 và trước Ke Jie, nhà vô địch thế giới, vào tháng 5 năm 2017. Chưa có chương trình nào từng đến gần việc đánh bại một bậc thầy của trò chơi này, chứ đừng nói đến nhà vô địch thế giới. Ngày nay, toàn bộ lĩnh vực RL đang sôi sục với những ý tưởng mới, với nhiều ứng dụng đa dạng.

Vậy DeepMind (được Google mua lại với giá hơn 500 triệu đô la vào năm 2014) đã đạt được tất cả những điều này như thế nào? Nhìn lại, có vẻ khá đơn giản: họ đã áp dụng sức mạnh của học sâu vào lĩnh vực học tăng cường, và nó đã hoạt động vượt xa những gì họ tưởng tượng. Trong chương này, tôi sẽ giải thích học tăng cường là gì và nó tốt ở điểm nào, sau đó trình bày hai trong số các kỹ thuật quan trọng nhất trong học tăng cường sâu: độ dốc chính sách (policy gradients) và mạng Q sâu (deep Q-networks), bao gồm thảo luận về các quá trình quyết định Markov. Hãy bắt đầu!

Học để tối ưu hóa phần thưởng

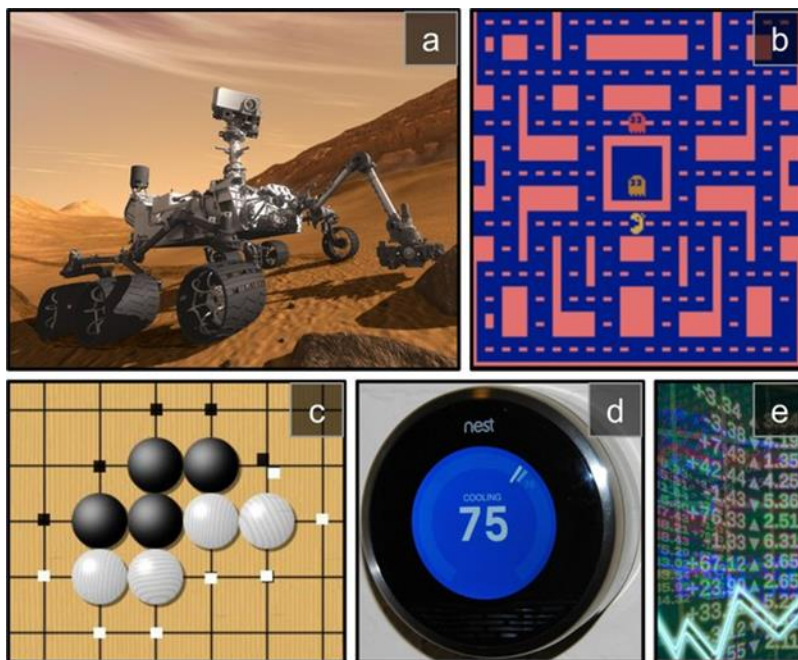
Trong học tăng cường, một tác nhân phần mềm thực hiện các quan sát và thực hiện các hành động trong một môi trường, và đổi lại nó nhận được phần thưởng từ môi trường. Mục tiêu của nó là học cách hành động theo cách sẽ tối đa hóa phần thưởng dự kiến của nó theo thời gian. Nếu bạn không ngại một chút nhân hóa, bạn có thể coi phần thưởng tích cực là khoái cảm và phần thưởng tiêu cực là nỗi đau (thuật ngữ “phần thưởng” hơi gây hiểu lầm trong trường hợp này). Tóm lại, tác nhân hành động trong môi trường và học bằng thử và sai để tối đa hóa khoái cảm và giảm thiểu nỗi đau.

Đây là một bối cảnh khá rộng, có thể áp dụng cho nhiều nhiệm vụ khác nhau. Dưới đây là một vài ví dụ (xem Hình 18-1):

- Tác nhân có thể là chương trình điều khiển robot. Trong trường hợp này, môi trường là thế giới thực, tác nhân quan sát môi trường thông qua một bộ cảm biến như camera và cảm biến chạm, và các hành động của nó bao gồm việc gửi tín hiệu để kích hoạt động cơ. Nó có thể được lập trình để nhận phần thưởng tích cực bất cứ khi nào nó tiếp cận đích đến, và phần thưởng tiêu cực bất cứ khi nào nó lãng phí thời gian hoặc đi sai hướng.

- Tác nhân có thể là chương trình điều khiển Ms. Pac-Man. Trong trường hợp này, môi trường là một mô phỏng của trò chơi Atari, các hành động là chín vị trí joystick có thể có (trên cùng bên trái, xuống, giữa, v.v.), các quan sát là ảnh chụp màn hình, và phần thưởng chỉ là điểm trò chơi.
- Tương tự, tác nhân có thể là chương trình chơi một trò chơi cờ bàn như Cờ vây. Nó chỉ nhận được phần thưởng nếu nó thắng.
- Tác nhân không nhất thiết phải điều khiển một vật thể di chuyển vật lý (hoặc ảo). Ví dụ, nó có thể là một bộ điều nhiệt thông minh, nhận phần thưởng tích cực bất cứ khi nào nó gần nhiệt độ mục tiêu và tiết kiệm năng lượng, và phần thưởng tiêu cực khi con người cần điều chỉnh nhiệt độ, do đó tác nhân phải học cách dự đoán nhu cầu của con người.
- Tác nhân có thể quan sát giá thị trường chứng khoán và quyết định mua hoặc bán bao nhiêu mỗi giây. Phần thưởng rõ ràng là lợi nhuận và thua lỗ tiền tệ.

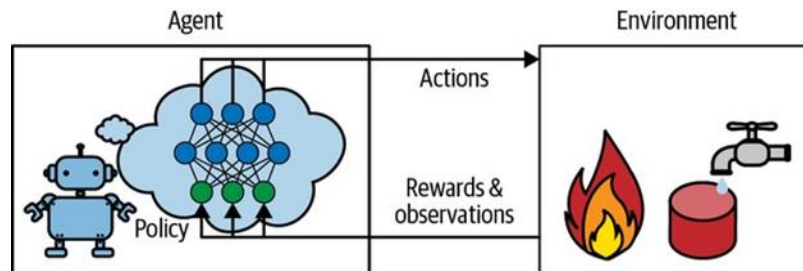
Lưu ý rằng có thể không có bất kỳ phần thưởng tích cực nào; ví dụ, tác nhân có thể di chuyển trong một mê cung, nhận phần thưởng tiêu cực ở mỗi bước thời gian, vì vậy tốt hơn là nó nên tìm lối ra càng nhanh càng tốt! Có nhiều ví dụ khác về các nhiệm vụ mà học tăng cường rất phù hợp, chẳng hạn như ô tô tự lái, hệ thống khuyến nghị, đặt quảng cáo trên trang web hoặc kiểm soát nơi hệ thống phân loại hình ảnh nên tập trung sự chú ý.



Hình 18-1. Các ví dụ về học tăng cường: (a) robot, (b) Ms. Pac-Man, (c) người chơi Cờ vây, (d) bộ điều nhiệt, (e) nhà giao dịch tự động

Tìm kiếm chính sách (Policy Search)

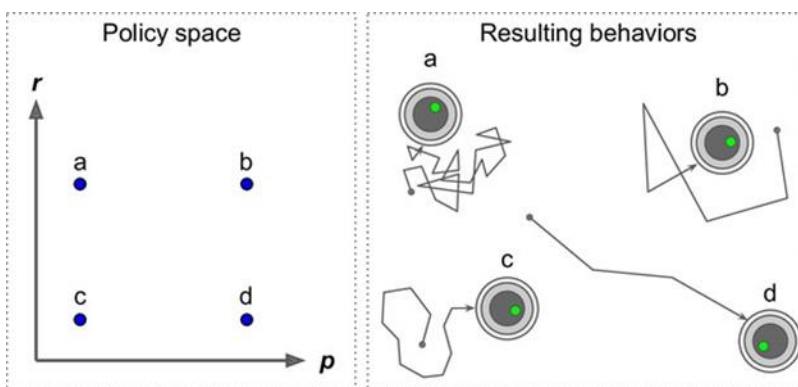
Thuật toán mà một tác nhân phần mềm sử dụng để xác định các hành động của nó được gọi là chính sách của nó. Chính sách có thể là một mạng nơ-ron nhận các quan sát làm đầu vào và xuất ra hành động cần thực hiện (xem Hình 18-2).



Hình 18-2. Học tăng cường sử dụng chính sách mạng nơ-ron

Chính sách có thể là bất kỳ thuật toán nào bạn có thể nghĩ ra, và nó không nhất thiết phải là xác định. Trên thực tế, trong một số trường hợp, nó thậm chí không cần phải quan sát môi trường! Ví dụ, hãy xem xét một robot hút bụi mà phần thưởng của nó là lượng bụi nó hút được trong 30 phút. Chính sách của nó có thể là di chuyển về phía trước với một xác suất p mỗi giây, hoặc xoay ngẫu nhiên sang trái hoặc phải với xác suất $1 - p$. Góc xoay sẽ là một góc ngẫu nhiên giữa $-r$ và $+r$. Vì chính sách này liên quan đến một số ngẫu nhiên, nó được gọi là chính sách ngẫu nhiên. Robot sẽ có một quỹ đạo thất thường, đảm bảo rằng nó cuối cùng sẽ đến bất kỳ nơi nào nó có thể tiếp cận và hút hết bụi. Câu hỏi là, nó sẽ hút được bao nhiêu bụi trong 30 phút?

Bạn sẽ huấn luyện một robot như vậy như thế nào? Chỉ có hai tham số chính sách mà bạn có thể điều chỉnh: xác suất p và phạm vi góc r . Một thuật toán học có thể là thử nhiều giá trị khác nhau cho các tham số này, và chọn sự kết hợp hoạt động tốt nhất (xem Hình 18-3). Đây là một ví dụ về tìm kiếm chính sách, trong trường hợp này sử dụng cách tiếp cận vét cạn. Khi không gian chính sách quá lớn (thường là như vậy), việc tìm một bộ tham số tốt theo cách này giống như tìm kim đáy bể. Một cách khác để khám phá không gian chính sách là sử dụng thuật toán di truyền. Ví dụ, bạn có thể tạo ngẫu nhiên thế hệ chính sách đầu tiên gồm 100 chính sách và thử chúng, sau đó “loại bỏ” 80 chính sách tệ nhất và để 20 chính sách sống sót tạo ra 4 con mỗi chính sách. Một con là một bản sao của bố mẹ nó cộng với một số biến thể ngẫu nhiên. Các chính sách sống sót cùng với con cái của chúng tạo thành thế hệ thứ hai. Bạn có thể tiếp tục lặp lại các thế hệ theo cách này cho đến khi bạn tìm thấy một chính sách tốt.



Hình 18-3. Bốn điểm trong không gian chính sách (trái) và hành vi tương ứng của tác nhân (phải)

Một cách tiếp cận khác là sử dụng các kỹ thuật tối ưu hóa, bằng cách đánh giá độ dốc của phần thưởng đối với các tham số chính sách, sau đó điều chỉnh các tham số này bằng cách đi theo độ dốc về phía phần thưởng cao hơn. Chúng ta sẽ thảo luận chi tiết hơn về cách tiếp cận này, được gọi là độ dốc chính sách (PG), sau trong chương này. Quay trở lại robot hút bụi, bạn có thể tăng nhẹ

p và đánh giá xem việc làm đó có làm tăng lượng bụi robot hút được trong 30 phút hay không; nếu có, thì tăng p thêm nữa, hoặc ngược lại giảm p . Chúng ta sẽ triển khai một thuật toán PG phổ biến bằng TensorFlow, nhưng trước khi làm vậy, chúng ta cần tạo một môi trường để tác nhân sống — vì vậy đã đến lúc giới thiệu OpenAI Gym.

Giới thiệu về OpenAI Gym

Một trong những thách thức của học tăng cường là để huấn luyện một tác nhân, trước tiên bạn cần có một môi trường hoạt động. Nếu bạn muốn lập trình một tác nhân sẽ học cách chơi một trò chơi Atari, bạn sẽ cần một trình mô phỏng trò chơi Atari. Nếu bạn muốn lập trình một robot đi bộ, thì môi trường là thế giới thực, và bạn có thể trực tiếp huấn luyện robot của mình trong môi trường đó. Tuy nhiên, điều này có những hạn chế: nếu robot rơi xuống vách đá, bạn không thể nhấp Hoàn tác. Bạn cũng không thể tăng tốc thời gian — thêm sức mạnh tính toán sẽ không làm robot di chuyển nhanh hơn — và nói chung quá tốn kém để huấn luyện 1.000 robot song song. Tóm lại, việc huấn luyện rất khó và chậm trong thế giới thực, vì vậy bạn thường cần một môi trường mô phỏng ít nhất để huấn luyện khởi động. Ví dụ, bạn có thể sử dụng một thư viện như PyBullet hoặc MuJoCo để mô phỏng vật lý 3D.

OpenAI Gym là một bộ công cụ cung cấp nhiều môi trường mô phỏng đa dạng (trò chơi Atari, trò chơi cờ bàn, mô phỏng vật lý 2D và 3D, v.v.) mà bạn có thể sử dụng để huấn luyện tác nhân, so sánh chúng hoặc phát triển các thuật toán RL mới.

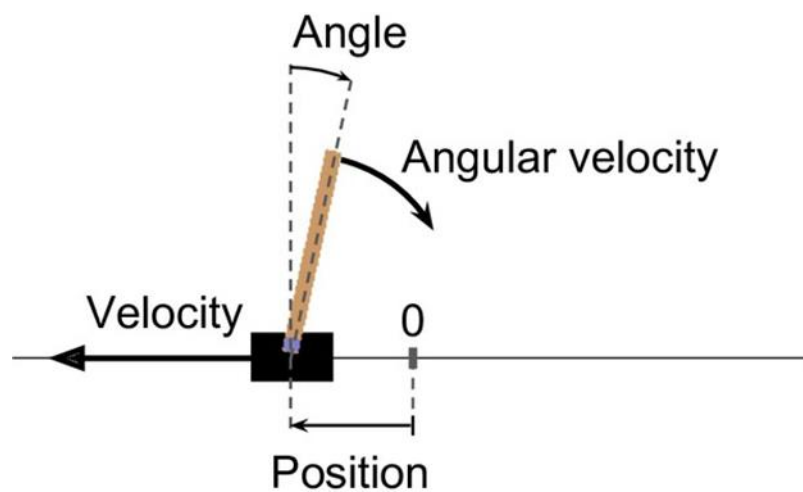
OpenAI Gym được cài đặt sẵn trên Colab, nhưng đó là một phiên bản cũ hơn, vì vậy bạn sẽ cần thay thế nó bằng phiên bản mới nhất. Bạn cũng cần cài đặt một vài phần phụ thuộc của nó. Nếu bạn đang viết mã trên máy của riêng mình thay vì Colab và bạn đã làm theo hướng dẫn cài đặt tại <https://homl.info/install>, thì bạn có thể bỏ qua bước này; nếu không, hãy nhập các lệnh này:

```
#Chỉ chạy các lệnh này trên Colab hoặc Kaggle!  
%pip install -q -U gym  
%pip install -q -U gym[classic_control,box2d,atari,accept-rom-license]
```

Lệnh %pip đầu tiên nâng cấp Gym lên phiên bản mới nhất. Tùy chọn -q có nghĩa là “quiet”: nó làm cho đầu ra ít dài dòng hơn. Tùy chọn -U có nghĩa là “upgrade”. Lệnh %pip thứ hai cài đặt các thư viện cần thiết để chạy các loại môi trường khác nhau. Điều này bao gồm các môi trường cổ điển từ lý thuyết điều khiển — khoa học về điều khiển hệ thống động lực — chẳng hạn như giữ thăng bằng một cây cột trên một chiếc xe đẩy. Nó cũng bao gồm các môi trường dựa trên thư viện Box2D — một công cụ vật lý 2D cho trò chơi. Cuối cùng, nó bao gồm các môi trường dựa trên Arcade Learning Environment (ALE), là một trình giả lập cho các trò chơi Atari 2600. Một số ROM trò chơi Atari được tải xuống tự động và bằng cách chạy mã này, bạn đồng ý với giấy phép ROM của Atari. Với những điều đó, bạn đã sẵn sàng sử dụng OpenAI Gym. Hãy nhập nó và tạo một môi trường:

```
env = gym.make("CartPole-v1", render_mode="rgb_array")
```

Ở đây, chúng ta đã tạo một môi trường CartPole. Đây là một mô phỏng 2D trong đó một chiếc xe đẩy có thể được tăng tốc sang trái hoặc phải để giữ thăng bằng một cây cột đặt trên đó (xem Hình 18-4). Đây là một nhiệm vụ điều khiển cổ điển.



Hình 18-4. Môi trường CartPole

Sau khi môi trường được tạo, bạn phải khởi tạo nó bằng phương thức `reset()`, tùy chọn chỉ định một hạt giống ngẫu nhiên. Điều này trả về quan sát đầu tiên. Các quan sát phụ thuộc vào loại môi trường. Đối với môi trường CartPole, mỗi quan sát là một mảng NumPy 1D chứa bốn số float đại diện cho vị trí ngang của xe đẩy (0.0 = trung tâm), vận tốc của nó (dương nghĩa là sang phải), góc của cây cột (0.0 = thẳng đứng) và vận tốc góc của nó (dương nghĩa là theo chiều kim đồng hồ). Phương thức `reset()` cũng trả về một từ điển có thể chứa thông tin bổ sung cụ thể cho môi trường. Điều này có thể hữu ích cho việc gỡ lỗi hoặc huấn luyện. Ví dụ, trong nhiều môi trường Atari, nó chứa số mạng còn lại. Tuy nhiên, trong môi trường CartPole, từ điển này trống.

```
>>> obs, info = env.reset(seed=42)
>>> obs
array([ 0.0273956 , -0.00611216, 0.03585979, 0.0197368 ], dtype=float32)
>>> info
{}
```

Hãy gọi phương thức `render()` để hiển thị môi trường này dưới dạng một hình ảnh. Vì chúng ta đã đặt `render_mode="rgb_array"` khi tạo môi trường, hình ảnh sẽ được trả về dưới dạng một mảng NumPy:

```
>>> img = env.render()
>>> img.shape # chiều cao, chiều rộng, kênh (3 = Đỏ, Xanh lá, Xanh dương)
(400, 600, 3)
```

Bạn có thể sử dụng hàm `imshow()` của Matplotlib để hiển thị hình ảnh này, như bình thường. Bây giờ hãy hỏi môi trường những hành động nào có thể thực hiện:

```
>>> env.action_space Discrete(2)
```

`Discrete(2)` có nghĩa là các hành động có thể là số nguyên 0 và 1, đại diện cho việc tăng tốc sang trái hoặc phải. Các môi trường khác có thể có các hành động rời rạc bổ sung, hoặc các loại hành động khác (ví dụ: liên tục). Vì cây cột đang nghiêng về bên phải (`obs[2] > 0`), hãy tăng tốc xe đẩy về bên phải:

```
>>> action = 1 # tăng tốc sang phải
>>> obs, reward, done, truncated, info = env.step(action)
>>> obs
array([ 0.02727336, 0.18847767, 0.03625453, -0.26141977], dtype=float32)
>>> reward 1.0
>>> done False
>>> truncated False
>>> info
{}
```

Phương thức `step()` thực hiện hành động mong muốn và trả về năm giá trị: `obs` Đây là quan sát mới. Xe đẩy hiện đang di chuyển về bên phải (`obs[1] > 0`). Cây cột vẫn nghiêng về bên phải (`obs[2] > 0`), nhưng vận tốc góc của nó hiện là âm (`obs[3] < 0`), vì vậy nó có thể sẽ nghiêng về bên trái sau bước tiếp theo.

`reward` Trong môi trường này, bạn nhận được phần thưởng 1.0 ở mỗi bước, bất kể bạn làm gì, vì vậy mục tiêu là giữ cho tập phim chạy càng lâu càng tốt.

`done` Giá trị này sẽ là `True` khi tập phim kết thúc. Điều này sẽ xảy ra khi cây cột nghiêng quá nhiều, hoặc đi ra khỏi màn hình, hoặc sau 200 bước (trong trường hợp sau, bạn đã thắng). Sau đó, môi trường phải được đặt lại trước khi có thể sử dụng lại.

`truncated` Giá trị này sẽ là `True` khi một tập phim bị gián đoạn sớm, ví dụ bởi một trình bao môi trường áp đặt số bước tối đa cho mỗi tập phim (xem tài liệu của Gym để biết thêm chi tiết về trình bao môi trường). Một số thuật toán RL xử lý các tập phim bị cắt cụt khác với các

tập phim kết thúc bình thường (tức là khi done là True), nhưng trong chương này chúng ta sẽ xử lý chúng giống hệt nhau.

info Từ điển cụ thể của môi trường này có thể cung cấp thông tin bổ sung, giống như cái được trả về bởi phương thức `reset()`.

Hãy mã hóa cứng một chính sách đơn giản là tăng tốc sang trái khi cây cột nghiêng về bên trái và tăng tốc sang phải khi cây cột nghiêng về bên phải. Chúng ta sẽ chạy chính sách này để xem phần thưởng trung bình nó nhận được trong 500 tập phim:

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1

totals = []
for episode in range(500):
    episode_rewards = 0
    obs, info = env.reset(seed=episode)
    for step in range(200):
        action = basic_policy(obs)
        obs, reward, done, truncated, info = env.step(action)
        episode_rewards += reward
        if done or truncated:
            break
    totals.append(episode_rewards)
```

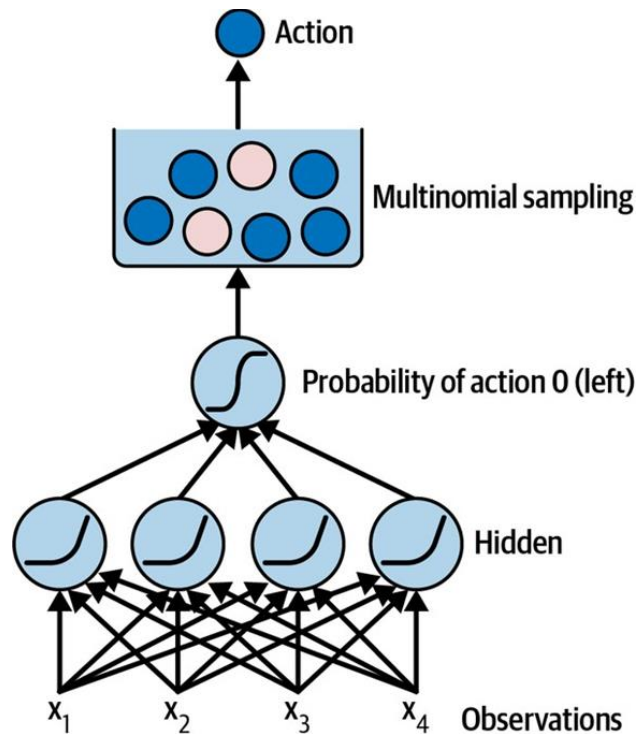
Mã này tự giải thích. Hãy xem kết quả:

```
>>> import numpy as np
>>> np.mean(totals), np.std(totals), min(totals), max(totals)
(41.698, 8.389445512070509, 24.0, 63.0)
```

Ngay cả với 500 lần thử, chính sách này không bao giờ giữ cây cột thẳng đứng được quá 63 bước liên tiếp. Không tốt lắm. Nếu bạn nhìn vào mô phỏng trong sổ ghi chép của chương này, bạn sẽ thấy rằng xe đẩy dao động sang trái và phải ngày càng mạnh cho đến khi cây cột nghiêng quá nhiều. Hãy xem liệu một mạng nơ-ron có thể đưa ra một chính sách tốt hơn không.

Các chính sách mạng nơ-ron

Hãy tạo một chính sách mạng nơ-ron. Mạng nơ-ron này sẽ nhận một quan sát làm đầu vào, và nó sẽ xuất ra hành động cần thực hiện, giống như chính sách chúng ta đã mã hóa cứng trước đó. Chính xác hơn, nó sẽ ước tính một xác suất cho mỗi hành động, và sau đó chúng ta sẽ chọn một hành động ngẫu nhiên, theo các xác suất ước tính (xem Hình 18-5). Trong trường hợp môi trường CartPole, chỉ có hai hành động có thể (trái hoặc phải), vì vậy chúng ta chỉ cần một nơ-ron đầu ra. Nó sẽ xuất ra xác suất p của hành động 0 (trái), và tất nhiên xác suất của hành động 1 (phải) sẽ là $1 - p$. Ví dụ, nếu nó xuất ra 0.7, thì chúng ta sẽ chọn hành động 0 với 70 xác suất, hoặc hành động 1 với 30 xác suất.



Hình 18-5. Chính sách mạng nơ-ron

Bạn có thể tự hỏi tại sao chúng ta lại chọn một hành động ngẫu nhiên dựa trên các xác suất được đưa ra bởi mạng nơ-ron, thay vì chỉ chọn hành động có điểm cao nhất. Cách tiếp cận này cho phép tác nhân tìm được sự cân bằng phù hợp giữa việc khám phá các hành động mới và khai thác các hành động đã biết là hoạt động tốt. Đây là một ví dụ tương tự: giả sử bạn lần đầu tiên đến một nhà hàng, và tất cả các món ăn đều trông hấp dẫn như nhau, vì vậy bạn ngẫu nhiên chọn một món. Nếu nó hóa ra là ngon, bạn có thể tăng xác suất bạn sẽ gọi món đó lần sau, nhưng bạn không nên tăng xác suất đó lên đến 100, nếu không bạn sẽ không bao giờ thử các món khác, một số trong đó có thể còn ngon hơn món bạn đã thử. Tình thế tiến thoái lưỡng nan khám phá/khai thác này là trung tâm trong học tăng cường. Cũng lưu ý rằng trong môi trường cụ thể này, các hành động và quan sát trong quá khứ có thể được bỏ qua một cách an toàn, vì mỗi quan sát chứa đầy đủ trạng thái của môi trường. Nếu có một trạng thái ẩn nào đó, thì bạn có thể cần phải xem xét cả các hành động và quan sát trong quá khứ. Ví dụ, nếu môi trường chỉ tiết lộ vị trí của xe đẩy mà không phải vận tốc của nó, bạn sẽ phải xem xét không chỉ quan sát hiện tại mà còn cả quan sát trước đó để ước tính vận tốc hiện tại. Một ví dụ khác là khi các quan sát bị nhiễu; trong trường hợp đó, bạn thường muốn sử dụng vài quan sát gần đây để ước tính trạng thái hiện tại có khả năng nhất. Vấn đề CartPole do đó đơn giản nhất có thể; các quan sát không bị nhiễu và chúng chứa đầy đủ trạng thái của môi trường. Đây là mã để xây dựng một chính sách mạng nơ-ron cơ bản bằng Keras:

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(5, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
```

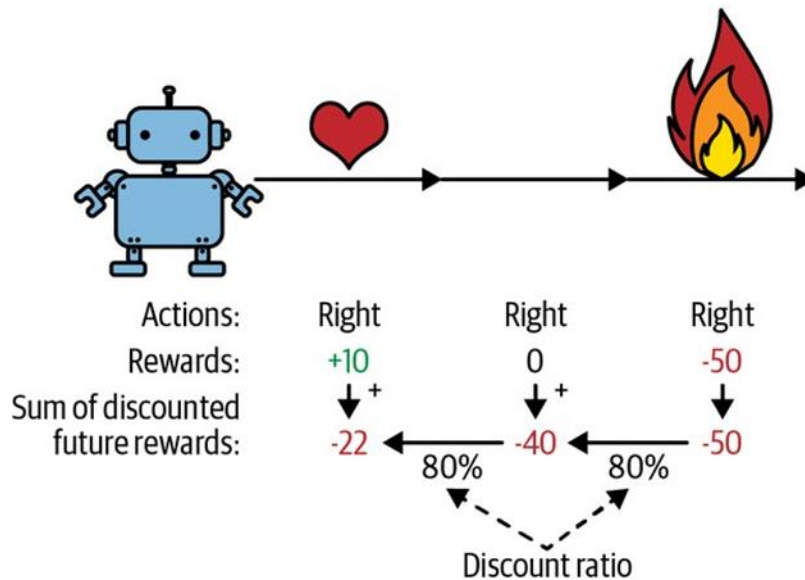

Chúng ta sử dụng một mô hình Sequential để định nghĩa mạng chính sách. Số lượng đầu vào là kích thước của không gian quan sát — trong trường hợp CartPole là 4 — và chúng ta chỉ có năm đơn vị ẩn vì đây là một nhiệm vụ khá đơn giản. Cuối cùng, chúng ta muốn xuất ra một xác suất duy nhất — xác suất đi sang trái — vì vậy chúng ta có một nơ-ron đầu ra duy nhất sử dụng hàm kích hoạt sigmoid. Nếu có nhiều hơn hai hành động có thể, sẽ có một nơ-ron đầu ra cho mỗi hành động, và chúng ta sẽ sử dụng hàm kích hoạt softmax thay thế. Được rồi, bây giờ chúng ta có một chính sách mạng nơ-ron sẽ nhận các quan sát và xuất ra xác suất hành động. Nhưng làm thế nào để chúng ta huấn luyện nó?

Đánh giá hành động: Bài toán phân bổ tín nhiệm

Nếu chúng ta biết hành động tốt nhất là gì ở mỗi bước, chúng ta có thể huấn luyện mạng nơ-ron như bình thường, bằng cách giảm thiểu entropy chéo giữa phân phối xác suất ước tính và phân phối xác suất mục tiêu. Đó sẽ chỉ là học có giám sát thông thường. Tuy nhiên, trong học tăng cường, hướng dẫn duy nhất mà tác nhân nhận được là thông qua phần thưởng, và phần thưởng thường thưa thớt và bị trì hoãn. Ví dụ, nếu tác nhân giữ thăng bằng được cây cột trong 100 bước, làm thế nào nó có thể biết hành động nào trong số 100 hành động nó đã thực hiện là tốt, và hành động nào là xấu? Tất cả những gì nó biết là cây cột đã rơi sau hành động cuối cùng, nhưng chắc chắn hành động cuối cùng này không hoàn toàn chịu trách nhiệm. Đây được gọi là **bài toán phân bổ tín nhiệm (credit assignment problem)**: khi tác nhân nhận được phần thưởng, rất khó để nó biết hành động nào nên được ghi công (hoặc đổ lỗi) cho phần thưởng đó. Hãy nghĩ về một con chó được thưởng sau nhiều giờ sau khi nó cư xử tốt; liệu nó có hiểu nó đang được thưởng vì điều gì không?

Để giải quyết vấn đề này, một chiến lược phổ biến là đánh giá một hành động dựa trên tổng của tất cả các phần thưởng đến sau nó, thường áp dụng một hệ số chiết khấu, *gamma* (gamma), ở mỗi bước. Tổng của các phần thưởng được chiết khấu này được gọi là **lợi tức (return)** của hành động. Xem xét ví dụ trong Hình 18-6.

Nếu một tác nhân quyết định đi sang phải ba lần liên tiếp và nhận được phần thưởng +10 sau bước đầu tiên, 0 sau bước thứ hai, và cuối cùng -50 sau bước thứ ba, thì giả sử chúng ta sử dụng hệ số chiết khấu $\gamma = 0.8$, hành động đầu tiên sẽ có lợi tức là $10 + \gamma \times 0 + \gamma^2 \times (-50) = -22$. Nếu hệ số chiết khấu gần 0, thì các phần thưởng trong tương lai sẽ không có nhiều giá trị so với các phần thưởng tức thời. Ngược lại, nếu hệ số chiết khấu gần 1, thì các phần thưởng ở xa trong tương lai sẽ có giá trị gần như phần thưởng tức thời. Các hệ số chiết khấu điển hình thay đổi từ 0.9 đến 0.99. Với hệ số chiết khấu 0.95, phần thưởng 13 bước trong tương lai có giá trị bằng khoảng một nửa phần thưởng tức thời (vì $0.95^{13} \approx 0.5$), trong khi với hệ số chiết khấu 0.99, phần thưởng 69 bước trong tương lai có giá trị bằng một nửa phần thưởng tức thời. Trong môi trường CartPole, các hành động có tác động khá ngắn hạn, vì vậy việc chọn hệ số chiết khấu là 0.95 có vẻ hợp lý.



Hình 18-6. Tính toán lợi tức của một hành động: tổng các phần thưởng tương lai được chiết khấu

Tất nhiên, một hành động tốt có thể được theo sau bởi một số hành động xấu khiến cây cột rơi nhanh chóng, dẫn đến hành động tốt nhận được lợi tức thấp. Tương tự, một diễn viên giỏi đôi khi có thể đóng vai chính trong một bộ phim tồi tệ. Tuy nhiên, nếu chúng ta chơi trò chơi đủ lần, trung bình các hành động tốt sẽ nhận được lợi tức cao hơn các hành động xấu. Chúng ta muốn ước tính một hành động tốt hơn hoặc tệ hơn bao nhiêu, so với các hành động khả thi khác, trung bình. Đây được gọi là **lợi thế hành động (action advantage)**. Để làm được điều này, chúng ta phải chạy nhiều tập và chuẩn hóa tất cả các lợi tức hành động, bằng cách trừ đi giá trị trung bình và chia cho độ lệch chuẩn. Sau đó, chúng ta có thể hợp lý giả định rằng các hành động có lợi thế âm là xấu trong khi các hành động có lợi thế dương là tốt. Được rồi, bây giờ chúng ta đã có cách để đánh giá từng hành động, chúng ta đã sẵn sàng huấn luyện tác nhân đầu tiên của mình bằng cách sử dụng độ dốc chính sách. Hãy xem cách thực hiện.

Độ dốc chính sách (Policy Gradients)

Như đã thảo luận trước đó, các thuật toán PG tối ưu hóa các tham số của một chính sách bằng cách đi theo độ dốc về phía phần thưởng cao hơn. Một lớp thuật toán PG phổ biến, được gọi là thuật toán REINFORCE, được Ronald Williams giới thiệu vào năm 1992. Đây là một biến thể phổ biến:

- Đầu tiên, hãy để chính sách mạng nơ-ron chơi trò chơi nhiều lần, và ở mỗi bước, tính toán các độ dốc sẽ làm cho hành động đã chọn có khả năng xảy ra cao hơn — nhưng đừng áp dụng các độ dốc này ngay lập tức.
- Sau khi bạn đã chạy một số tập, hãy tính lợi thế của mỗi hành động, bằng cách sử dụng phương pháp được mô tả trong phần trước.

- Nếu lợi thế của một hành động là dương, điều đó có nghĩa là hành động đó có lẽ là tốt, và bạn muốn áp dụng các độ dốc đã tính toán trước đó để làm cho hành động đó có khả năng được chọn trong tương lai cao hơn. Tuy nhiên, nếu lợi thế của hành động là âm, điều đó có nghĩa là hành động đó có lẽ là xấu, và bạn muốn áp dụng các độ dốc ngược lại để làm cho hành động này ít khả năng xảy ra hơn trong tương lai. Giải pháp là nhân mỗi vector độ dốc với lợi thế tương ứng của hành động.
- Cuối cùng, tính giá trị trung bình của tất cả các vector độ dốc thu được, và sử dụng nó để thực hiện một bước giảm độ dốc.

Hãy sử dụng Keras để triển khai thuật toán này. Chúng ta sẽ huấn luyện chính sách mạng nơ-ron mà chúng ta đã xây dựng trước đó để nó học cách giữ thăng bằng cây cột trên xe đẩy. Đầu tiên, chúng ta cần một hàm sẽ chơi một bước. Hiện tại chúng ta sẽ giả vờ rằng bất kỳ hành động nào nó thực hiện đều là hành động đúng để chúng ta có thể tính toán tổn thất và độ dốc của nó. Các độ dốc này sẽ chỉ được lưu trong một thời gian, và chúng ta sẽ sửa đổi chúng sau này tùy thuộc vào hành động đó hóa ra tốt hay xấu:

```
def play_one_step(env, obs, model, loss_fn):
    with tf.GradientTape() as tape:
        left_proba = model(obs[np.newaxis])
        action = (tf.random.uniform([1, 1]) > left_proba)
        y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
        loss = tf.reduce_mean(loss_fn(y_target, left_proba))

    grads = tape.gradient(loss, model.trainable_variables)
    obs, reward, done, truncated, info = env.step(int(action))
    return obs, reward, done, truncated, grads
```

Hãy cùng xem qua hàm này:

- Trong khối GradientTape (xem Chương 12), chúng ta bắt đầu bằng cách gọi mô hình, đưa cho nó một quan sát duy nhất. Chúng ta định hình lại quan sát để nó trở thành một lô chứa một thể hiện duy nhất, vì mô hình mong đợi một lô. Điều này xuất ra xác suất đi sang trái.
- Tiếp theo, chúng ta lấy mẫu một số float ngẫu nhiên giữa 0 và 1, và chúng ta kiểm tra xem nó có lớn hơn `left_proba` hay không. Hành động sẽ là `False` với xác suất `left_proba`, hoặc `True` với xác suất `1 - left_proba`. Khi chúng ta chuyển đổi giá trị Boolean này thành một số nguyên, hành động sẽ là 0 (trái) hoặc 1 (phải) với các xác suất thích hợp.
- Bây giờ chúng ta định nghĩa xác suất mục tiêu của việc đi sang trái: đó là 1 trừ đi hành động (được chuyển đổi thành một số float). Nếu hành động là 0 (trái), thì xác suất mục tiêu của việc đi sang trái sẽ là 1. Nếu hành động là 1 (phải), thì xác suất mục tiêu sẽ là 0.
- Sau đó, chúng ta tính toán tổn thất bằng cách sử dụng hàm tổn thất đã cho, và chúng ta sử dụng bằng để tính toán độ dốc của tổn thất đối với các biến có thể huấn luyện

của mô hình. Một lần nữa, các độ dốc này sẽ được điều chỉnh sau này, trước khi chúng ta áp dụng chúng, tùy thuộc vào hành động đó hóa ra tốt hay xấu.

- Cuối cùng, chúng ta thực hiện hành động đã chọn, và chúng ta trả về quan sát mới, phần thưởng, liệu tập phim đã kết thúc hay chưa, liệu nó có bị cắt cụt hay không, và tất nhiên là các độ dốc mà chúng ta vừa tính toán.

Bây giờ hãy tạo một hàm khác sẽ dựa vào hàm `play_one_step()` để chơi nhiều tập, trả về tất cả các phần thưởng và độ dốc cho mỗi tập và mỗi bước:

```
def play_multiple_episodes(env, n_episodes, n_max_steps, model, loss_fn):
    all_rewards = []
    all_grads = []

    for episode in range(n_episodes):
        current_rewards = []
        current_grads = []
        obs, info = env.reset()

        for step in range(n_max_steps):
            obs, reward, done, truncated, grads = play_one_step(env, obs, model, loss_fn)
            current_rewards.append(reward)
            current_grads.append(grads)

            if done or truncated:
                break

        all_rewards.append(current_rewards)
        all_grads.append(current_grads)

    return all_rewards, all_grads
```

Mã này trả về một danh sách các danh sách phần thưởng: một danh sách phần thưởng cho mỗi tập, chứa một phần thưởng cho mỗi bước. Nó cũng trả về một danh sách các danh sách độ dốc: một danh sách độ dốc cho mỗi tập, mỗi danh sách chứa một bộ độ dốc cho mỗi bước và mỗi bộ chứa một tensor độ dốc cho mỗi biến có thể huấn luyện. Thuật toán sẽ sử dụng hàm `play_multiple_episodes()` để chơi trò chơi nhiều lần (ví dụ: 10 lần), sau đó nó sẽ quay lại và xem tất cả các phần thưởng, chiết khấu chúng và chuẩn hóa chúng. Để làm điều đó, chúng ta cần thêm một vài hàm; hàm đầu tiên sẽ tính tổng các phần thưởng tương lai được chiết khấu ở mỗi bước, và hàm thứ hai sẽ chuẩn hóa tất cả các phần thưởng được chiết khấu này (tức là lợi tức) trên nhiều tập bằng cách trừ đi giá trị trung bình và chia cho độ lệch chuẩn:

```
def discount_rewards(rewards, discount_factor):
    discounted = np.array(rewards)
    for step in range(len(rewards) - 2, -1, -1):
        discounted[step] += discounted[step + 1] * discount_factor
    return discounted
```

```
def discount_and_normalize_rewards(all_rewards, discount_factor):
    all_discounted_rewards = [discount_rewards(rewards, discount_factor)
                              for rewards in all_rewards]
    flat_rewards = np.concatenate(all_discounted_rewards)
    reward_mean = flat_rewards.mean()
    reward_std = flat_rewards.std()

    return [(discounted_rewards - reward_mean) / reward_std for discounted_re
wards in all_discounted_rewards]
```

Hãy kiểm tra xem điều này có hoạt động không:

```
>>> discount_rewards([10, 0, -50], discount_factor=0.8)
array([-22, -40, -50])

>>> discount_and_normalize_rewards([[10, 0, -50], [10, 20]],
...                               discount_factor=0.8)
...
[array([-0.28435071, -0.86597718, -1.18910299]),
 array([1.26665318, 1.0727777 ])]
```

Cuộc gọi tới `discount_rewards()` trả về chính xác những gì chúng ta mong đợi (xem Hình 18-6). Bạn có thể xác minh rằng hàm `discount_and_normalize_rewards()` thực sự trả về lợi thế hành động đã chuẩn hóa cho mỗi hành động trong cả hai tập. Lưu ý rằng tập đầu tiên tệ hơn nhiều so với tập thứ hai, vì vậy tất cả các lợi thế đã chuẩn hóa của nó đều là số âm; tất cả các hành động từ tập đầu tiên sẽ được coi là xấu, và ngược lại tất cả các hành động từ tập thứ hai sẽ được coi là tốt. Chúng ta gần như đã sẵn sàng để chạy thuật toán! Bây giờ hãy định nghĩa các siêu tham số. Chúng ta sẽ chạy 150 lần lặp huấn luyện, chơi 10 tập cho mỗi lần lặp, và mỗi tập sẽ kéo dài tối đa 200 bước. Chúng ta sẽ sử dụng hệ số chiết khấu là 0.95 :

```
n_iterations = 150
n_episodes_per_update = 10
n_max_steps = 200
discount_factor = 0.95
```

Chúng ta cũng cần một trình tối ưu hóa và hàm tổn thất. Một trình tối ưu hóa Nadam thông thường với tốc độ học 0.01 sẽ hoạt động tốt, và chúng ta sẽ sử dụng hàm tổn thất entropy chéo nhị phân vì chúng ta đang huấn luyện một bộ phân loại nhị phân (có hai hành động có thể — trái hoặc phải):

```
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
loss_fn = tf.keras.losses.binary_crossentropy
```

Bây giờ chúng ta đã sẵn sàng để xây dựng và chạy vòng lặp huấn luyện!

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)
    all_final_rewards = discount_and_normalize_rewards(all_rewards,
```

```

discount_factor)

all_mean_grads = []

for var_index in range(len(model.trainable_variables)):
    mean_grads = tf.reduce_mean(
        [final_reward * all_grads[episode_index][step][var_index]
         for episode_index, final_rewards in enumerate(all_final_rewards)
         for step, final_reward in enumerate(final_rewards)], axis=0)
    all_mean_grads.append(mean_grads)
optimizer.apply_gradients(zip(all_mean_grads, model.trainable_variables))

```

Hãy cùng xem qua mã này:

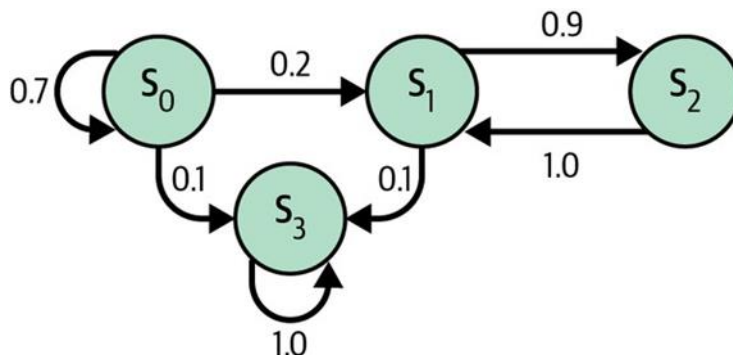
- Ở mỗi lần lặp huấn luyện, vòng lặp này gọi hàm `play_multiple_episodes()`, hàm này chơi 10 tập và trả về phần thưởng và độ dốc cho mỗi bước trong mỗi tập.
- Sau đó, chúng ta gọi hàm `discount_and_normalize_rewards()` để tính lợi thế chuẩn hóa của mỗi hành động, được gọi là `final_reward` trong mã này. Điều này cung cấp một thước đo về mức độ tốt hay xấu của mỗi hành động, nhìn lại.
- Tiếp theo, chúng ta xem xét từng biến có thể huấn luyện, và đối với mỗi biến đó, chúng ta tính toán giá trị trung bình có trọng số của độ dốc cho biến đó trên tất cả các tập và tất cả các bước, được trọng số bằng `final_reward`.
- Cuối cùng, chúng ta áp dụng các độ dốc trung bình này bằng cách sử dụng trình tối ưu hóa: các biến có thể huấn luyện của mô hình sẽ được điều chỉnh, và hy vọng chính sách sẽ tốt hơn một chút.

Và chúng ta đã hoàn tất! Mã này sẽ huấn luyện chính sách mạng nơ-ron, và nó sẽ học cách giữ thăng bằng cây cột trên xe đẩy một cách thành công. Phần thưởng trung bình mỗi tập sẽ đạt rất gần 200. Theo mặc định, đó là giá trị tối đa cho môi trường này. Thành công! Thuật toán độ dốc chính sách đơn giản mà chúng ta vừa huấn luyện đã giải quyết nhiệm vụ CartPole, nhưng nó sẽ không mở rộng tốt cho các nhiệm vụ lớn hơn và phức tạp hơn. Thật vậy, nó rất kém hiệu quả về mẫu, nghĩa là nó cần khám phá trò chơi trong một thời gian rất dài trước khi có thể đạt được tiến bộ đáng kể. Điều này là do nó phải chạy nhiều tập để ước tính lợi thế của mỗi hành động, như chúng ta đã thấy. Tuy nhiên, nó là nền tảng của các thuật toán mạnh mẽ hơn, chẳng hạn như thuật toán actor-critic (mà chúng ta sẽ thảo luận ngắn gọn ở cuối chương này).

Bây giờ chúng ta sẽ xem xét một nhóm thuật toán phổ biến khác. Trong khi các thuật toán PG trực tiếp cố gắng tối ưu hóa chính sách để tăng phần thưởng, các thuật toán chúng ta sẽ khám phá bây giờ ít trực tiếp hơn: tác nhân học cách ước tính lợi tức dự kiến cho mỗi trạng thái, hoặc cho mỗi hành động trong mỗi trạng thái, sau đó nó sử dụng kiến thức này để quyết định cách hành động. Để hiểu các thuật toán này, trước tiên chúng ta phải xem xét các quá trình quyết định Markov (MDPs).

Các quá trình quyết định Markov (Markov Decision Processes)

Vào đầu thế kỷ 20, nhà toán học Andrey Markov đã nghiên cứu các quá trình ngẫu nhiên không có bộ nhớ, được gọi là **chuỗi Markov (Markov chains)**. Một quá trình như vậy có một số trạng thái cố định, và nó ngẫu nhiên tiến hóa từ trạng thái này sang trạng thái khác ở mỗi bước. Xác suất để nó tiến hóa từ trạng thái s sang trạng thái s' là cố định, và nó chỉ phụ thuộc vào cặp (s, s') , không phụ thuộc vào các trạng thái trong quá khứ. Đây là lý do tại sao chúng ta nói rằng hệ thống không có bộ nhớ. Hình 18-7 cho thấy một ví dụ về một chuỗi Markov với bốn trạng thái.

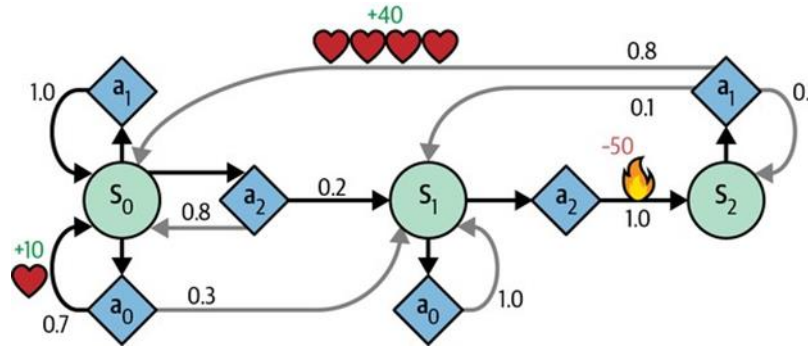


Hình 18-7. Ví dụ về một chuỗi Markov

Giả sử quá trình bắt đầu ở trạng thái s_0 , và có 70% khả năng nó sẽ giữ nguyên trạng thái đó ở bước tiếp theo. Cuối cùng nó sẽ phải rời khỏi trạng thái đó và không bao giờ quay lại, bởi vì không có trạng thái nào khác chỉ ngược lại s_0 . Nếu nó chuyển sang trạng thái s_1 , nó sẽ rất có khả năng chuyển sang trạng thái s_2 (90% xác suất), sau đó ngay lập tức quay lại trạng thái s_1 (100% xác suất). Nó có thể luân phiên một số lần giữa hai trạng thái này, nhưng cuối cùng nó sẽ rơi vào trạng thái s_3 và ở đó mãi mãi, vì không có lối thoát: đây được gọi là **trạng thái kết thúc (terminal state)**. Các chuỗi Markov có thể có động lực học rất khác nhau, và chúng được sử dụng nhiều trong nhiệt động lực học, hóa học, thống kê, và nhiều hơn nữa.

Các quá trình quyết định Markov được Richard Bellman mô tả lần đầu tiên vào những năm 1950. Chúng giống với chuỗi Markov, nhưng có một sự khác biệt: ở mỗi bước, một tác nhân có thể chọn một trong số nhiều hành động khả thi, và xác suất chuyển đổi phụ thuộc vào hành động đã chọn. Hơn nữa, một số chuyển đổi trạng thái trả về một phần thưởng (tích cực hoặc tiêu cực), và mục tiêu của tác nhân là tìm một chính sách sẽ tối đa hóa phần thưởng theo thời gian.

Ví dụ, MDP được biểu diễn trong Hình 18-8 có ba trạng thái (được biểu thị bằng các vòng tròn) và tối đa ba hành động rời rạc có thể ở mỗi bước (được biểu thị bằng các hình thoi).



Hình 18-8. Ví dụ về một quá trình quyết định Markov

Nếu nó bắt đầu ở trạng thái s_0 , tác nhân có thể chọn giữa các hành động a_0 , a_1 , hoặc a_2 . Nếu nó chọn hành động a_1 , nó chỉ giữ nguyên trạng thái s_0 một cách chắc chắn, và không có bất kỳ phần thưởng nào. Do đó nó có thể quyết định ở đó mãi mãi nếu muốn. Nhưng nếu nó chọn hành động a_0 , nó có 70% xác suất nhận được phần thưởng +10 và giữ nguyên trạng thái s_0 . Sau đó nó có thể thử đi thử lại để nhận được càng nhiều phần thưởng càng tốt, nhưng đến một lúc nào đó nó sẽ kết thúc ở trạng thái s_1 . Ở trạng thái s_1 nó chỉ có hai hành động có thể: a_0 hoặc a_2 . Nó có thể chọn ở lại bằng cách liên tục chọn hành động a_0 , hoặc nó có thể chọn chuyển sang trạng thái s_2 và nhận một phần thưởng âm -50 (ouch). Ở trạng thái s_2 nó không có lựa chọn nào khác ngoài việc thực hiện hành động a_1 , điều này rất có thể sẽ đưa nó trở lại trạng thái s_0 , nhận được phần thưởng +40 trên đường đi. Bạn hình dung được rồi chứ. Bằng cách nhìn vào MDP này, bạn có thể đoán chiến lược nào sẽ mang lại nhiều phần thưởng nhất theo thời gian không? Ở trạng thái s_0 rõ ràng hành động a_0 là lựa chọn tốt nhất, và ở trạng thái s_2 tác nhân không có lựa chọn nào khác ngoài việc thực hiện hành động a_1 , nhưng ở trạng thái s_1 không rõ liệu tác nhân nên ở lại (a_0) hay đi qua lửa (a_2).

Bellman đã tìm ra một cách để ước tính **giá trị trạng thái tối ưu (optimal state value)** của bất kỳ trạng thái s nào, được ký hiệu là $V^*(s)$, là tổng của tất cả các phần thưởng tương lai được chiết khấu mà tác nhân có thể mong đợi trung bình sau khi nó đạt đến trạng thái đó, giả sử nó hành động tối ưu. Ông đã chỉ ra rằng nếu tác nhân hành động tối ưu, thì **phương trình tối ưu Bellman (Bellman optimality equation)** sẽ áp dụng (xem Phương trình 18-1). Phương trình đệ quy này nói rằng nếu tác nhân hành động tối ưu, thì giá trị tối ưu của trạng thái hiện tại bằng phần thưởng nó sẽ nhận được trung bình sau khi thực hiện một hành động tối ưu, cộng với giá trị tối ưu dự kiến của tất cả các trạng thái tiếp theo có thể mà hành động này có thể dẫn đến. Phương trình 18-1. Phương trình tối ưu Bellman

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V^*(s')]$$

Trong phương trình này:

- $T(s, a, s')$ là xác suất chuyển đổi từ trạng thái s sang trạng thái s' , với điều kiện tác nhân đã chọn hành động a . Ví dụ, trong Hình 18-8, $T(s_2, a_1, s_0) = 0.8$.

- $R(s, a, s')$ là phần thưởng mà tác nhân nhận được khi nó đi từ trạng thái s sang trạng thái s' , với điều kiện tác nhân đã chọn hành động a . Ví dụ, trong Hình 18-8, $R(s_2, a_1, s_0) = +40$.
- γ là hệ số chiết khấu.

Phương trình này dẫn trực tiếp đến một thuật toán có thể ước tính chính xác giá trị trạng thái tối ưu của mọi trạng thái có thể: đầu tiên khởi tạo tất cả các ước tính giá trị trạng thái bằng không, và sau đó cập nhật chúng lặp đi lặp lại bằng thuật toán **lặp giá trị (value iteration algorithm)** (xem Phương trình 18-2). Một kết quả đáng chú ý là, nếu có đủ thời gian, các ước tính này được đảm bảo hội tụ về các giá trị trạng thái tối ưu, tương ứng với chính sách tối ưu. Phương trình 18-2. Thuật toán lặp giá trị

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V_k(s')]$$

Trong phương trình này, $V_k(s)$ là giá trị ước tính của trạng thái s tại lần lặp thứ k của thuật toán.

Biết các giá trị trạng thái tối ưu có thể hữu ích, đặc biệt để đánh giá một chính sách, nhưng nó không cung cấp cho chúng ta chính sách tối ưu cho tác nhân. May mắn thay, Bellman đã tìm ra một thuật toán rất tương tự để ước tính các **giá trị trạng thái-hành động tối ưu (optimal state-action values)**, thường được gọi là **giá trị Q (Q-values)** (giá trị chất lượng). Giá trị Q tối ưu của cặp trạng thái-hành động (s, a) , được ký hiệu là $Q^*(s, a)$, là tổng của các phần thưởng tương lai được chiết khấu mà tác nhân có thể mong đợi trung bình sau khi nó đạt đến trạng thái s và chọn hành động a , nhưng trước khi nó thấy kết quả của hành động này, giả sử nó hành động tối ưu sau hành động đó.

Hãy xem nó hoạt động như thế nào. Một lần nữa, bạn bắt đầu bằng cách khởi tạo tất cả các ước tính giá trị Q bằng không, sau đó bạn cập nhật chúng bằng thuật toán **lặp giá trị Q (Q-value iteration algorithm)** (xem Phương trình 18-3).

Phương trình 18-3. Thuật toán lặp giá trị Q

Công thức 18-3: Thuật toán lặp giá trị Q

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot \max_{a'} Q_k(s', a')] \forall (s, a)$$

Một khi bạn có các giá trị Q tối ưu, việc xác định chính sách tối ưu, được ký hiệu là $\pi^*(s)$, là rất đơn giản: khi tác nhân ở trạng thái s , nó nên chọn hành động có giá trị Q cao nhất cho trạng thái đó:

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Hãy áp dụng thuật toán này cho MDP được biểu diễn trong Hình 18-8. Đầu tiên, chúng ta cần định nghĩa MDP:

```

transition_probabilities = [ # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [ # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]

```

Ví dụ, để biết xác suất chuyển đổi từ s_2 sang s_0 sau khi thực hiện hành động a_1 , chúng ta sẽ tra `transition_probabilities[2][1][0]` (là 0.8). Tương tự, để nhận được phần thưởng tương ứng, chúng ta sẽ tra `rewards[2][1][0]` (là +40). Và để nhận danh sách các hành động có thể ở s_2 , chúng ta sẽ tra `possible_actions[2]` (trong trường hợp này, chỉ có hành động a_1 là có thể). Tiếp theo, chúng ta phải khởi tạo tất cả các giá trị Q bằng không (trừ các hành động không thể, đối với chúng ta đặt giá trị Q là $-\infty$):

```

Q_values = np.full((3, 3), -np.inf) # -np.inf for impossible actions

for state, actions in enumerate(possible_actions):
    Q_values[state, actions] = 0.0 # for all possible actions

```

Bây giờ hãy chạy thuật toán lặp giá trị Q. Nó áp dụng Phương trình 18-3 lặp đi lặp lại, cho tất cả các giá trị Q, cho mọi trạng thái và mọi hành động có thể:

```

gamma = 0.90 # the discount factor

for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                transition_probabilities[s][a][sp]
                * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)])

```

Vậy là xong! Các giá trị Q thu được trông như thế này:

```

>>> Q_values
array([[18.91891892, 17.02702702, 13.62162162],
       [ 0.          , -inf, -4.87971488],
       [-inf, 50.13365013, -inf]])

```

Ví dụ, khi tác nhân ở trạng thái s_0 và nó chọn hành động a_1 , tổng dự kiến của các phần thưởng tương lai được chiết khấu xấp xỉ 17.0. Đối với mỗi trạng thái, chúng ta có thể tìm hành động có giá trị Q cao nhất:

```
>>> Q_values.argmax(axis=1) # optimal action for each state
array([0, 0, 1])
```

Điều này cung cấp cho chúng ta chính sách tối ưu cho MDP này khi sử dụng hệ số chiết khấu là 0.90 : ở trạng thái s_0 chọn hành động a_0 , ở trạng thái s_1 chọn hành động a_0 (tức là giữ nguyên), và ở trạng thái s_2 chọn hành động a_1 (hành động duy nhất có thể). Thật thú vị, nếu chúng ta tăng hệ số chiết khấu lên 0.95 , chính sách tối ưu thay đổi: ở trạng thái s_1 hành động tốt nhất trở thành a_2 (đi qua lửa!). Điều này có ý nghĩa vì bạn càng coi trọng phần thưởng trong tương lai, bạn càng sẵn lòng chịu đựng một số nỗi đau hiện tại để đổi lấy lời hứa về hạnh phúc trong tương lai.

Học chênh lệch thời gian (Temporal Difference Learning)

Các bài toán học tăng cường với các hành động rời rạc thường có thể được mô hình hóa dưới dạng các quá trình quyết định Markov (MDPs), nhưng ban đầu tác nhân không biết xác suất chuyển đổi là gì (nó không biết

$T(s, a, s')$), và nó cũng không biết phần thưởng sẽ là gì ($R(s, a, s')$). Nó phải trải nghiệm mỗi trạng thái và mỗi lần chuyển đổi ít nhất một lần để biết phần thưởng, và nó phải trải nghiệm chúng nhiều lần nếu muốn có ước tính hợp lý về xác suất chuyển đổi.

Thuật toán học chênh lệch thời gian (TD) rất giống với thuật toán lặp giá trị Q, nhưng được điều chỉnh để tính đến việc tác nhân chỉ có kiến thức một phần về MDP. Nói chung, chúng ta giả định rằng tác nhân ban đầu chỉ biết các trạng thái và hành động có thể, và không có gì hơn. Tác nhân sử dụng một

chính sách khám phá (exploration policy) — ví dụ, một chính sách hoàn toàn ngẫu nhiên — để khám phá MDP, và khi nó tiến triển, thuật toán học TD cập nhật các ước tính của các giá trị trạng thái dựa trên các chuyển đổi và phần thưởng thực sự được quan sát (xem Phương trình 18-4).

Công thức 18-4: Thuật toán học TD $V_{k+1}(s)(1 - \gamma)V_k(s) + (\gamma + V_k(s'))$ hoặc tương đương:

$$V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$$

Trong phương trình này:

- α là **tốc độ học** (ví dụ: 0.01).
- $r + \gamma \cdot V_k(s')$ được gọi là **mục tiêu TD** (TD target).
- $\delta_k(s, r, s')$ được gọi là **sai số TD** (TD error).

Một cách viết ngắn gọn hơn cho dạng đầu tiên của phương trình này là sử dụng ký hiệu $a \leftarrow_b c$, có nghĩa là $a_{k+1} \leftarrow (1 - b_k)a_k + b_k c$. Vì vậy, dòng đầu tiên của Công thức 18-4 có thể được viết lại như sau:

$$V(s) \leftarrow_{\alpha} r + \gamma \cdot V(s')$$

Đối với mỗi trạng thái s , thuật toán này theo dõi một giá trị trung bình chạy của các phần thưởng tức thời mà tác nhân nhận được khi rời khỏi trạng thái đó, cộng với các phần thưởng mà nó mong đợi nhận được sau này, giả sử nó hành động tối ưu.

Học Q (Q-Learning)

Tương tự, thuật toán **học Q (Q-learning)** là một sự điều chỉnh của thuật toán lặp giá trị Q cho tình huống mà xác suất chuyển đổi và phần thưởng ban đầu không xác định (xem Phương trình 18-5). Học Q hoạt động bằng cách quan sát một tác nhân chơi (ví dụ: ngẫu nhiên) và dần dần cải thiện các ước tính của các giá trị Q của nó. Khi nó có ước tính giá trị Q chính xác (hoặc đủ gần), thì chính sách tối ưu chỉ đơn giản là chọn hành động có giá trị Q cao nhất (tức là chính sách tham lam).

Phương trình 18-5. Thuật toán học Q

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left(r + \gamma \cdot \max_{a'} Q(s', a') \right)$$

Đối với mỗi cặp trạng thái-hành động

(s, a) , thuật toán này theo dõi một giá trị trung bình chạy của các phần thưởng r mà tác nhân nhận được khi rời trạng thái s với hành động a , cộng với tổng các phần thưởng tương lai được chiết khấu mà nó mong đợi nhận được. Để ước tính tổng này, chúng ta lấy giá trị tối đa của các ước tính giá trị Q cho trạng thái tiếp theo

s' , vì chúng ta giả định rằng chính sách mục tiêu sẽ hành động tối ưu từ đó trở đi.

Hãy triển khai thuật toán học Q. Đầu tiên, chúng ta sẽ cần làm cho một tác nhân khám phá môi trường. Để làm điều này, chúng ta cần một hàm

step để tác nhân có thể thực hiện một hành động và nhận được trạng thái và phần thưởng kết quả:

```
def step(state, action):
    probas = transition_probabilities[state][action]
    next_state = np.random.choice([0, 1, 2], p=probas)
    reward = rewards[state][action][next_state]
    return next_state, reward
```

Bây giờ hãy triển khai chính sách khám phá của tác nhân. Vì không gian trạng thái khá nhỏ, một chính sách ngẫu nhiên đơn giản sẽ đủ. Nếu chúng ta chạy thuật toán đủ lâu, tác nhân sẽ ghé thăm mọi trạng thái nhiều lần, và nó cũng sẽ thử mọi hành động có thể nhiều lần:

```
def exploration_policy(state):
    return np.random.choice(possible_actions[state])
```

Tiếp theo, sau khi chúng ta khởi tạo các giá trị Q giống như trước đó, chúng ta đã sẵn sàng chạy thuật toán học Q với suy giảm tốc độ học (sử dụng lập lịch lũy thừa, được giới thiệu trong Chương 11):

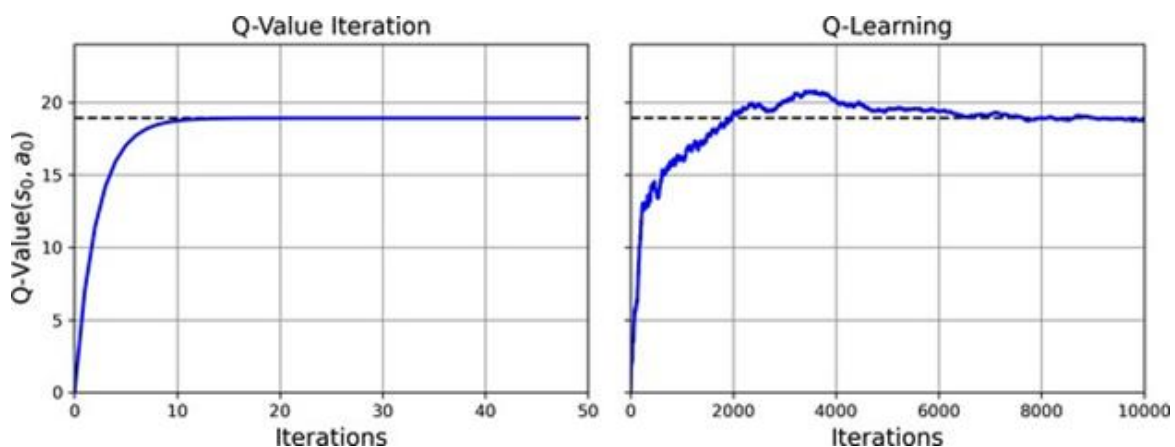
```

alpha0 = 0.05 # initial learning rate
decay = 0.005 # learning rate decay
gamma = 0.90 # discount factor
state = 0 # initial state

for iteration in range(10_000):
    action = exploration_policy(state)
    next_state, reward = step(state, action)
    next_value = Q_values[next_state].max() # greedy policy at the next step
    alpha = alpha0 / (1 + iteration * decay)
    Q_values[state, action] *= 1 - alpha
    Q_values[state, action] += alpha * (reward + gamma * next_value)
    state = next_state

```

Thuật toán này sẽ hội tụ đến các giá trị Q tối ưu, nhưng nó sẽ mất nhiều lần lặp, và có thể khá nhiều lần điều chỉnh siêu tham số. Như bạn có thể thấy trong Hình 18-9, thuật toán lặp giá trị Q (trái) hội tụ rất nhanh, trong vòng chưa đầy 20 lần lặp, trong khi thuật toán học Q (phải) mất khoảng 8.000 lần lặp để hội tụ. Rõ ràng, việc không biết xác suất chuyển đổi hoặc phần thưởng làm cho việc tìm chính sách tối ưu khó hơn đáng kể!



Hình 18-9. Đường cong học của thuật toán lặp giá trị Q so với thuật toán học Q

Thuật toán học Q được gọi là

thuật toán off-policy (off-policy algorithm) vì chính sách đang được huấn luyện không nhất thiết là chính sách được sử dụng trong quá trình huấn luyện. Ví dụ, trong mã chúng ta vừa chạy, chính sách được thực thi (chính sách khám phá) hoàn toàn ngẫu nhiên, trong khi chính sách đang được huấn luyện không bao giờ được sử dụng. Sau khi huấn luyện, chính sách tối ưu tương ứng với việc chọn hành động có giá trị Q cao nhất một cách có hệ thống. Ngược lại, thuật toán độ dốc chính sách là một

thuật toán on-policy (on-policy algorithm): nó khám phá thế giới bằng cách sử dụng chính sách đang được huấn luyện. Điều hơi đáng ngạc nhiên là học Q có khả năng học chính sách tối ưu chỉ bằng cách quan sát một tác nhân hành động ngẫu nhiên. Hãy tưởng tượng việc học chơi gôn khi giáo viên của bạn là một con khỉ bị bịt mắt. Chúng ta có thể làm tốt hơn không?

Các chính sách khám phá (Exploration Policies)

Tất nhiên, học Q chỉ có thể hoạt động nếu chính sách khám phá khám phá MDP đủ kỹ lưỡng. Mặc dù một chính sách hoàn toàn ngẫu nhiên được đảm bảo cuối cùng sẽ ghé thăm mọi trạng thái và mọi lần chuyển đổi nhiều lần, nhưng nó có thể mất một thời gian cực kỳ dài để làm được điều đó. Do đó, một lựa chọn tốt hơn là sử dụng

chính sách ϵ -tham lam (ϵ -greedy policy) (ϵ là epsilon): ở mỗi bước, nó hành động ngẫu nhiên với xác suất ϵ , hoặc tham lam với xác suất $1 - \epsilon$ (tức là chọn hành động có giá trị Q cao nhất). Ưu điểm của chính sách

ϵ -tham lam (so với một chính sách hoàn toàn ngẫu nhiên) là nó sẽ dành ngày càng nhiều thời gian để khám phá các phần thú vị của môi trường, khi các ước tính giá trị Q ngày càng tốt hơn, trong khi vẫn dành một chút thời gian để ghé thăm các vùng không xác định của MDP. Khá phổ biến khi bắt đầu với một giá trị

ϵ cao (ví dụ: 1.0) và sau đó giảm dần nó (ví dụ: xuống 0.05).

Ngoài ra, thay vì chỉ dựa vào cơ hội để khám phá, một cách tiếp cận khác là khuyến khích chính sách khám phá thử các hành động mà nó chưa thử nhiều trước đây. Điều này có thể được triển khai dưới dạng một phần thưởng được thêm vào các ước tính giá trị Q, như thể hiện trong Phương trình 18-6.

Công thức 18-6: Q-learning sử dụng hàm thăm dò

$$Q(s, a) \leftarrow_{\alpha} r + \gamma \cdot \max_{a'} f(Q(s', a'), N(s', a'))$$

Trong phương trình này:

- $N(s', a')$ đếm số lần hành động a' đã được chọn trong trạng thái s' .
- $f(Q, N)$ là một **hàm thăm dò**, ví dụ như $f(Q, N) = Q + \frac{\kappa}{1+N}$, trong đó κ là một **siêu tham số tò mò** (curiosity hyperparameter) đo lường mức độ tác nhân bị thu hút bởi những điều chưa biết.

Học Q xấp xỉ và Học Q sâu (Approximate Q-Learning and Deep Q-Learning)

Vấn đề chính với học Q là nó không mở rộng tốt cho các MDP lớn (hoặc thậm chí trung bình) với nhiều trạng thái và hành động. Ví dụ, giả sử bạn muốn sử dụng học Q để huấn luyện một tác nhân chơi Ms. Pac-Man (xem Hình 18-1). Có khoảng 150 viên thức ăn mà Ms. Pac-Man có thể ăn, mỗi viên có thể có hoặc không có (tức là đã ăn). Vì vậy, số lượng trạng thái có thể lớn hơn

$2^{150} \approx 10^{45}$. Và nếu bạn thêm tất cả các tổ hợp vị trí có thể cho tất cả các bóng ma và Ms. Pac-Man, số lượng trạng thái có thể trở nên lớn hơn số nguyên tử trên hành tinh của chúng ta, vì vậy hoàn toàn không có cách nào bạn có thể theo dõi ước tính cho từng giá trị Q riêng lẻ.

Giải pháp là tìm một hàm

$Q_{\theta}(s, a)$ xấp xỉ giá trị Q của bất kỳ cặp trạng thái-hành động (s, a) nào bằng cách sử dụng một số lượng tham số có thể quản lý được (được cho bởi vector tham số θ). Đây được gọi là

học Q xấp xỉ (approximate Q-learning). Trong nhiều năm, người ta khuyến nghị sử dụng các kết hợp tuyến tính của các đặc trưng được tạo thủ công được trích xuất từ trạng thái (ví dụ: khoảng cách của các bóng ma gần nhất, hướng của chúng, v.v.) để ước tính giá trị Q, nhưng vào năm 2013, DeepMind đã chỉ ra rằng việc sử dụng mạng nơ-ron sâu có thể hoạt động tốt hơn nhiều, đặc biệt đối với các vấn đề phức tạp, và nó không yêu cầu bất kỳ kỹ thuật đặc trưng nào. Một DNN được sử dụng để ước tính giá trị Q được gọi là

mạng Q sâu (deep Q-network - DQN), và việc sử dụng DQN cho học Q xấp xỉ được gọi là **học Q sâu (deep Q-learning)**.

Bây giờ, làm thế nào chúng ta có thể huấn luyện một DQN? Chà, hãy xem xét giá trị Q xấp xỉ được tính toán bởi DQN cho một cặp trạng thái-hành động

(s, a) nhất định. Nhờ Bellman, chúng ta biết rằng chúng ta muốn giá trị Q xấp xỉ này càng gần càng tốt với phần thưởng

r mà chúng ta thực sự quan sát được sau khi thực hiện hành động a ở trạng thái s , cộng với giá trị được chiết khấu của việc chơi tối ưu từ đó trở đi. Để ước tính tổng các phần thưởng tương lai được chiết khấu này, chúng ta có thể chỉ cần thực thi DQN trên trạng thái tiếp theo

s' , cho tất cả các hành động có thể a' . Chúng ta nhận được một giá trị Q tương lai xấp xỉ cho mỗi hành động có thể. Sau đó, chúng ta chọn giá trị cao nhất (vì chúng ta giả định rằng chúng ta sẽ chơi tối ưu) và chiết khấu nó, và điều này cung cấp cho chúng ta một ước tính tổng các phần thưởng tương lai được chiết khấu. Bằng cách cộng phần thưởng

r và ước tính giá trị được chiết khấu trong tương lai, chúng ta nhận được **giá trị Q mục tiêu (target Q-value)** $y(s, a)$ cho cặp trạng thái-hành động (s, a) , như thể hiện trong Phương trình 18-7.

Phương trình 18-7. Giá trị Q mục tiêu

$$y(s, a) = r + \gamma \cdot \max_{a'} Q_{\theta}(s', a')$$

Với giá trị Q mục tiêu này, chúng ta có thể chạy một bước huấn luyện bằng cách sử dụng bất kỳ thuật toán giảm độ dốc nào. Cụ thể, chúng ta thường cố gắng giảm thiểu sai số bình phương giữa giá trị Q ước tính

$Q_{\theta}(s, a)$ và giá trị Q mục tiêu $y(s, a)$, hoặc hàm Huber để giảm độ nhạy của thuật toán đối với các sai số lớn. Và đó là thuật toán học Q sâu! Hãy xem cách triển khai nó để giải quyết môi trường CartPole.

Triển khai học Q sâu

Điều đầu tiên chúng ta cần là một mạng Q sâu. Về lý thuyết, chúng ta cần một mạng nơ-ron nhận một cặp trạng thái-hành động làm đầu vào, và xuất ra một giá trị Q xấp xỉ. Tuy nhiên, trên thực tế, hiệu quả hơn nhiều khi sử dụng một mạng nơ-ron chỉ nhận một trạng thái làm đầu vào, và xuất ra một giá trị Q xấp xỉ cho mỗi hành động có thể. Để giải quyết môi trường CartPole, chúng ta không cần một mạng nơ-ron quá phức tạp; một vài lớp ẩn là đủ:

```
input_shape = [4] # == env.observation_space.shape
n_outputs = 2 # == env.action_space.n

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    tf.keras.layers.Dense(32, activation="elu"),
    tf.keras.layers.Dense(n_outputs)
])
```

Để chọn một hành động bằng cách sử dụng DQN này, chúng ta chọn hành động có giá trị Q dự đoán lớn nhất. Để đảm bảo rằng tác nhân khám phá môi trường, chúng ta sẽ sử dụng chính sách

epsilon-tham lam (tức là, chúng ta sẽ chọn một hành động ngẫu nhiên với xác suất *epsilon*):

```
def epsilon_greedy_policy(state, epsilon=0):
    if np.random.rand() < epsilon:
        return np.random.randint(n_outputs) # random action
    Q_values = model.predict(state[np.newaxis], verbose=0)[0]
    return Q_values.argmax() # optimal action according to the DQN
```

Thay vì huấn luyện DQN chỉ dựa trên các kinh nghiệm gần đây nhất, chúng ta sẽ lưu trữ tất cả các kinh nghiệm trong một **bộ đệm phát lại (replay buffer)** (hoặc bộ nhớ phát lại), và chúng ta sẽ lấy mẫu một lô huấn luyện ngẫu nhiên từ đó ở mỗi lần lặp huấn luyện. Điều này giúp giảm các mối tương quan giữa các kinh nghiệm trong một lô huấn luyện, điều này hỗ trợ rất nhiều cho việc huấn luyện. Để làm điều này, chúng ta sẽ chỉ sử dụng một hàng đợi hai đầu (deque):

```
from collections import deque

replay_buffer = deque(maxlen=2000)
```

Mỗi kinh nghiệm sẽ bao gồm sáu yếu tố: một trạng thái

s , hành động a mà tác nhân đã thực hiện, phần thưởng r thu được, trạng thái tiếp theo s' mà nó đạt được, một Boolean chỉ ra liệu tập phim đã kết thúc tại thời điểm đó (done), và cuối cùng là một Boolean khác chỉ ra liệu tập phim có bị cắt cụt tại thời điểm đó hay không. Chúng ta sẽ cần một hàm nhỏ để lấy mẫu một lô kinh nghiệm ngẫu nhiên từ bộ đệm phát lại. Nó sẽ trả về sáu mảng NumPy tương ứng với sáu yếu tố kinh nghiệm:

```
def sample_experiences(batch_size):
    indices = np.random.randint(len(replay_buffer), size=batch_size)
    batch = [replay_buffer[index] for index in indices]
    return [
        np.array([experience[field_index]
                  for experience in batch])
        for field_index in range(6)
    ] # [states, actions, rewards, next_states, done, truncateds]
```

Hãy tạo thêm một hàm sẽ chơi một bước duy nhất bằng cách sử dụng chính sách

epsilon -tham lam, sau đó lưu trữ kinh nghiệm thu được vào bộ đệm phát lại:

```
def play_one_step(env, state, epsilon):
    action = epsilon_greedy_policy(state, epsilon)
    next_state, reward, done, truncated, info = env.step(action)
    replay_buffer.append((state, action, reward, next_state, done, truncated)
    )
    return next_state, reward, done, truncated, info
```

Cuối cùng, hãy tạo một hàm cuối cùng sẽ lấy mẫu một lô kinh nghiệm từ bộ đệm phát lại và huấn luyện DQN bằng cách thực hiện một bước giảm độ dốc duy nhất trên lô này:

```
batch_size = 32
discount_factor = 0.95

optimizer = tf.keras.optimizers.Nadam(learning_rate=1e-2)
loss_fn = tf.keras.losses.mean_squared_error

def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, done, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0)
    max_next_Q_values = next_Q_values.max(axis=1)
    runs = 1.0 - (done | truncateds) # episode is not done or truncated
    target_Q_values = rewards + runs * discount_factor * max_next_Q_values
    target_Q_values = target_Q_values.reshape(-1, 1)

    mask = tf.one_hot(actions, n_outputs)
    with tf.GradientTape() as tape:
        all_Q_values = model(states)
        Q_values = tf.reduce_sum(all_Q_values * mask, axis=1, keepdims=True)
        loss = tf.reduce_mean(loss_fn(target_Q_values, Q_values))

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Đây là những gì đang diễn ra trong mã này:

- Đầu tiên, chúng ta định nghĩa một số siêu tham số, và chúng ta tạo trình tối ưu hóa và hàm tổn thất.
- Sau đó, chúng ta tạo hàm `training_step()`. Nó bắt đầu bằng cách lấy mẫu một lô kinh nghiệm, sau đó nó sử dụng DQN để dự đoán giá trị Q cho mỗi hành động có thể trong trạng thái tiếp theo của mỗi kinh nghiệm. Vì chúng ta giả định rằng tác nhân sẽ chơi tối ưu, chúng ta chỉ giữ giá trị Q tối đa cho mỗi trạng thái tiếp theo. Tiếp theo, chúng ta sử dụng Phương trình 18-7 để tính toán giá trị Q mục tiêu cho cặp trạng thái-hành động của mỗi kinh nghiệm.
- Chúng ta muốn sử dụng DQN để tính toán giá trị Q cho mỗi cặp trạng thái-hành động đã trải nghiệm, nhưng DQN cũng sẽ xuất ra các giá trị Q cho các hành động khả thi khác, không chỉ cho hành động thực sự được tác nhân chọn. Vì vậy, chúng ta cần che đi tất cả các giá trị Q mà chúng ta không cần. Hàm `tf.one_hot()` giúp chuyển đổi một mảng các chỉ số hành động thành một mặt nạ như vậy. Ví dụ, nếu ba kinh nghiệm đầu tiên chứa các hành động 1,1,0 tương ứng, thì mặt nạ sẽ bắt đầu bằng `[[0,1], [0,1], [1,0], ...]`. Sau đó chúng ta có thể nhân đầu ra của DQN với mặt nạ này, và điều này sẽ làm cho tất cả các giá trị Q mà chúng ta không muốn trở thành 0. Sau đó chúng ta tính tổng theo trục 1 để loại bỏ tất cả các số không, chỉ giữ lại các giá trị Q của các cặp trạng thái-hành động đã trải nghiệm. Điều này cung cấp cho chúng ta tensor `Q_values`, chứa một giá trị Q được dự đoán cho mỗi kinh nghiệm trong lô.
- Tiếp theo, chúng ta tính toán tổn thất: đó là sai số bình phương trung bình giữa các giá trị Q mục tiêu và dự đoán cho các cặp trạng thái-hành động đã trải nghiệm.
- Cuối cùng, chúng ta thực hiện một bước giảm độ dốc để giảm thiểu tổn thất đối với các biến có thể huấn luyện của mô hình.

Đây là phần khó nhất. Bây giờ việc huấn luyện mô hình rất đơn giản:

```
for episode in range(600):
    obs, info = env.reset()
    for step in range(200):
        epsilon = max(1 - episode / 500, 0.01)

        obs, reward, done, truncated, info = play_one_step(env, obs, epsilon)
        if done or truncated:
            break

    if episode > 50:
        training_step(batch_size)
```

Chúng ta chạy 600 tập, mỗi tập tối đa 200 bước. Ở mỗi bước, chúng ta đầu tiên tính toán giá trị `epsilon` cho chính sách

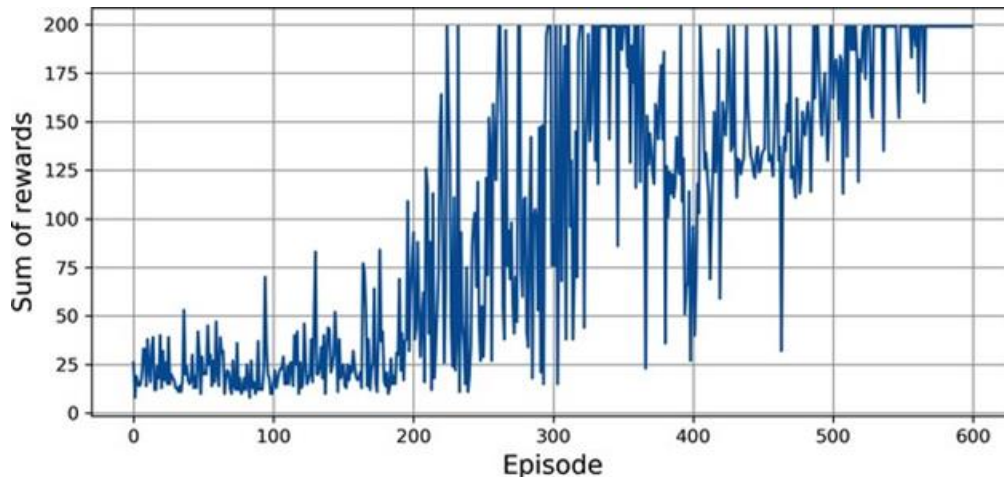
epsilon-tham lam: nó sẽ đi từ 1 xuống 0.01, tuyến tính, trong vòng chưa đầy 500 tập. Sau đó chúng ta gọi hàm

`play_one_step()`, hàm này sẽ sử dụng chính sách *epsilon*-tham lam để chọn một hành động, sau đó thực hiện nó và ghi lại kinh nghiệm vào bộ đệm phát lại. Nếu tập phim đã hoàn

thành hoặc bị cắt cụt, chúng ta thoát khỏi vòng lặp. Cuối cùng, nếu chúng ta đã qua tập 50, chúng ta gọi hàm

`training_step()` để huấn luyện mô hình trên một lô được lấy mẫu từ bộ đệm phát lại. Lý do chúng ta chơi nhiều tập mà không huấn luyện là để cho bộ đệm phát lại có thời gian để lấp đầy (nếu chúng ta không đợi đủ, sẽ không có đủ sự đa dạng trong bộ đệm phát lại). Và đó là nó: chúng ta vừa triển khai thuật toán học Q sâu!

Hình 18-10 cho thấy tổng phần thưởng mà tác nhân nhận được trong mỗi tập.



Hình 18-10. Đường cong học của thuật toán học Q sâu

Như bạn có thể thấy, thuật toán mất một thời gian để bắt đầu học bất cứ điều gì, một phần vì

epsilon rất cao ở ban đầu. Sau đó, tiến độ của nó thất thường: nó lần đầu tiên đạt phần thưởng tối đa vào khoảng tập 220, nhưng ngay lập tức giảm, sau đó nảy lên nảy xuống một vài lần, và ngay sau đó có vẻ như nó đã ổn định gần phần thưởng tối đa, vào khoảng tập 320, điểm số của nó lại giảm mạnh. Điều này được gọi là **quên thảm khốc (catastrophic forgetting)**, và đó là một trong những vấn đề lớn mà hầu hết các thuật toán RL đều gặp phải: khi tác nhân khám phá môi trường, nó cập nhật chính sách của mình, nhưng những gì nó học được ở một phần của môi trường có thể làm hỏng những gì nó đã học trước đó ở các phần khác của môi trường. Các kinh nghiệm khá tương quan, và môi trường học tập liên tục thay đổi — điều này không lý tưởng cho giảm độ dốc! Nếu bạn tăng kích thước của bộ đệm phát lại, thuật toán sẽ ít bị vấn đề này hơn. Điều chỉnh tốc độ học cũng có thể giúp ích. Nhưng sự thật là, học tăng cường rất khó: việc huấn luyện thường không ổn định, và bạn có thể cần thử nhiều giá trị siêu tham số và hạt giống ngẫu nhiên trước khi tìm thấy một sự kết hợp hoạt động tốt. Ví dụ, nếu bạn thử thay đổi hàm kích hoạt từ “*elu*” sang “*relu*”, hiệu suất sẽ thấp hơn nhiều.

Bạn có thể tự hỏi tại sao chúng ta không vẽ đồ thị tổn thất. Hóa ra tổn thất là một chỉ số kém về hiệu suất của mô hình. Tổn thất có thể giảm, nhưng tác nhân có thể hoạt động tệ hơn (ví dụ: điều này có thể xảy ra khi tác nhân bị kẹt trong một vùng nhỏ của môi trường, và DQN bắt

đầu overfitting vùng này). Ngược lại, tổn thất có thể tăng, nhưng tác nhân có thể hoạt động tốt hơn (ví dụ: nếu DQN đang đánh giá thấp các giá trị Q và nó bắt đầu tăng dự đoán của mình một cách chính xác, tác nhân có thể sẽ hoạt động tốt hơn, nhận được nhiều phần thưởng hơn, nhưng tổn thất có thể tăng vì DQN cũng đặt các mục tiêu, vốn cũng sẽ lớn hơn). Vì vậy, tốt hơn là vẽ đồ thị phần thưởng.

Thuật toán học Q sâu cơ bản mà chúng ta đã sử dụng cho đến nay sẽ quá không ổn định để học cách chơi các trò chơi Atari. Vậy DeepMind đã làm điều đó như thế nào? Chà, họ đã điều chỉnh thuật toán!

Các biến thể của học Q sâu (Deep Q-Learning Variants)

Hãy cùng xem xét một vài biến thể của thuật toán học Q sâu có thể ổn định và tăng tốc quá trình huấn luyện.

Mục tiêu giá trị Q cố định (Fixed Q-value Targets)

Trong thuật toán học Q sâu cơ bản, mô hình được sử dụng cả để đưa ra dự đoán và để thiết lập các mục tiêu của riêng nó. Điều này có thể dẫn đến một tình huống tương tự như một con chó đuổi theo cái đuôi của chính nó. Vòng lặp phản hồi này có thể làm cho mạng không ổn định: nó có thể phân kỳ, dao động, đóng băng, v.v. Để giải quyết vấn đề này, trong bài báo năm 2013 của họ, các nhà nghiên cứu của DeepMind đã sử dụng hai DQN thay vì một: một là **mô hình trực tuyến (online model)**, học ở mỗi bước và được sử dụng để di chuyển tác nhân, và một là **mô hình mục tiêu (target model)** chỉ được sử dụng để định nghĩa các mục tiêu. Mô hình mục tiêu chỉ là một bản sao của mô hình trực tuyến:

```
target = tf.keras.models.clone_model(model) # clone the model's architecture
target.set_weights(model.get_weights()) # copy the weights
```

Sau đó, trong hàm `training_step()`, chúng ta chỉ cần thay đổi một dòng để sử dụng mô hình mục tiêu thay vì mô hình trực tuyến khi tính toán các giá trị Q của các trạng thái tiếp theo:

```
next_Q_values = target.predict(next_states, verbose=0)
```

Cuối cùng, trong vòng lặp huấn luyện, chúng ta phải sao chép các trọng số của mô hình trực tuyến sang mô hình mục tiêu, theo các khoảng thời gian đều đặn (ví dụ: cứ sau 50 tập):

```
if episode % 50 == 0:
    target.set_weights(model.get_weights())
```

Vì mô hình mục tiêu được cập nhật ít thường xuyên hơn nhiều so với mô hình trực tuyến, các mục tiêu giá trị Q ổn định hơn, vòng lặp phản hồi mà chúng ta đã thảo luận trước đó được làm suy yếu, và các tác động của nó ít nghiêm trọng hơn. Cách tiếp cận này là một trong những đóng góp chính của các nhà nghiên cứu DeepMind trong bài báo năm 2013 của họ, cho phép các tác nhân học chơi các trò chơi Atari từ các pixel thô. Để ổn định quá trình huấn luyện, họ đã sử dụng tốc độ học rất nhỏ là

0.00025, họ chỉ cập nhật mô hình mục tiêu sau mỗi 10.000 bước (thay vì 50), và họ đã sử dụng một bộ đệm phát lại rất lớn gồm 1 triệu kinh nghiệm. Họ giảm epsilon rất chậm, từ 1

xuống 0.1 trong 1 triệu bước, và họ để thuật toán chạy trong 50 triệu bước. Hơn nữa, DQN của họ là một mạng tích chập sâu. Bây giờ hãy cùng xem xét một biến thể DQN khác đã đánh bại hiệu suất tốt nhất một lần nữa.

DQN kép (Double DQN)

Trong một bài báo năm 2015, các nhà nghiên cứu của DeepMind đã điều chỉnh thuật toán DQN của họ, tăng hiệu suất và phần nào ổn định quá trình huấn luyện. Họ gọi biến thể này là **DQN kép (double DQN)**. Bản cập nhật này dựa trên quan sát rằng mạng mục tiêu có xu hướng đánh giá quá cao các giá trị Q. Thật vậy, giả sử tất cả các hành động đều tốt như nhau: các giá trị Q được ước tính bởi mô hình mục tiêu phải giống hệt nhau, nhưng vì chúng là các xấp xỉ, một số có thể lớn hơn một chút so với những cái khác, hoàn toàn do ngẫu nhiên. Mô hình mục tiêu sẽ luôn chọn giá trị Q lớn nhất, giá trị này sẽ lớn hơn một chút so với giá trị Q trung bình, rất có thể đánh giá quá cao giá trị Q thực (một chút giống như đếm chiều cao của con sóng ngẫu nhiên cao nhất khi đo độ sâu của một cái hồ bơi). Để khắc phục điều này, các nhà nghiên cứu đã đề xuất sử dụng mô hình trực tuyến thay vì mô hình mục tiêu khi chọn các hành động tốt nhất cho các trạng thái tiếp theo, và chỉ sử dụng mô hình mục tiêu để ước tính các giá trị Q cho các hành động tốt nhất này. Đây là hàm `training_step()` đã được cập nhật:

```
def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0) # ≠ target.predict
    ()
    best_next_actions = next_Q_values.argmax(axis=1)
    next_mask = tf.one_hot(best_next_actions, n_outputs).numpy()
    max_next_Q_values = (target.predict(next_states, verbose=0) * next_mask).
    sum(axis=1)
    [...] # the rest is the same as earlier
```

Chỉ vài tháng sau, một cải tiến khác cho thuật toán DQN đã được đề xuất; chúng ta sẽ xem xét nó tiếp theo.

Phát lại kinh nghiệm ưu tiên (Prioritized Experience Replay)

Thay vì lấy mẫu kinh nghiệm đồng đều từ bộ đệm phát lại, tại sao không lấy mẫu các kinh nghiệm quan trọng thường xuyên hơn? Ý tưởng này được gọi là **lấy mẫu quan trọng (importance sampling - IS)** hoặc **phát lại kinh nghiệm ưu tiên (prioritized experience replay - PER)**, và nó được giới thiệu trong một bài báo năm 2015 bởi các nhà nghiên cứu của DeepMind (một lần nữa!).

Cụ thể hơn, các kinh nghiệm được coi là “quan trọng” nếu chúng có khả năng dẫn đến tiến bộ học nhanh chóng. Nhưng làm thế nào chúng ta có thể ước tính điều này? Một cách tiếp cận hợp lý là đo độ lớn của sai số TD $\delta = r + \gamma V(s') - V(s)$. Sai số TD lớn cho thấy rằng một chuyển đổi (s, a, s') rất bất ngờ, và do đó có lẽ đáng học hỏi từ. Khi một kinh nghiệm được ghi vào bộ đệm phát lại, ưu tiên của nó được đặt thành một giá trị rất lớn, để đảm bảo rằng nó được lấy mẫu ít nhất một lần. Tuy nhiên, một khi nó được lấy mẫu (và mỗi khi nó được lấy mẫu), sai số TD δ được tính toán, và ưu tiên của kinh nghiệm này

được đặt thành $p = |\delta|$ (cộng với một hằng số nhỏ để đảm bảo rằng mọi kinh nghiệm đều có xác suất được lấy mẫu khác không). Xác suất P lấy mẫu một kinh nghiệm với ưu tiên p tỷ lệ thuận với p^{zeta} , trong đó $zeta$ là một siêu tham số kiểm soát mức độ tham lam mà chúng ta muốn lấy mẫu quan trọng: khi $zeta = 0$, chúng ta chỉ nhận được lấy mẫu đồng đều, và khi $zeta = 1$, chúng ta nhận được lấy mẫu quan trọng hoàn chỉnh. Trong bài báo, các tác giả đã sử dụng $zeta = 0.6$, nhưng giá trị tối ưu sẽ phụ thuộc vào nhiệm vụ.

Tuy nhiên, có một điều đáng chú ý: vì các mẫu sẽ bị thiên vị theo hướng các kinh nghiệm quan trọng, chúng ta phải bù đắp cho sự thiên vị này trong quá trình huấn luyện bằng cách giảm trọng số các kinh nghiệm theo mức độ quan trọng của chúng, nếu không mô hình sẽ chỉ overfitting các kinh nghiệm quan trọng. Nói rõ hơn, chúng ta muốn các kinh nghiệm quan trọng được lấy mẫu thường xuyên hơn, nhưng điều này cũng có nghĩa là chúng ta phải cho chúng một trọng số thấp hơn trong quá trình huấn luyện. Để làm điều này, chúng ta định nghĩa trọng số huấn luyện của mỗi kinh nghiệm là $w = (nP)^{-beta}$, trong đó n là số lượng kinh nghiệm trong bộ đệm phát lại, và $beta$ là một siêu tham số kiểm soát mức độ chúng ta muốn bù đắp cho sự thiên vị lấy mẫu quan trọng (0 có nghĩa là không bù đắp gì cả, trong khi 1 có nghĩa là bù đắp hoàn toàn). Trong bài báo, các tác giả đã sử dụng

$beta = 0.4$ ở đầu quá trình huấn luyện và tăng tuyến tính nó lên $beta = 1$ vào cuối quá trình huấn luyện. Một lần nữa, giá trị tối ưu sẽ phụ thuộc vào nhiệm vụ, nhưng nếu bạn tăng một cái, bạn thường sẽ muốn tăng cái kia. Bây giờ hãy cùng xem xét một biến thể quan trọng cuối cùng của thuật toán DQN.

DQN đấu (Dueling DQN)

Thuật toán **DQN đấu (dueling DQN - DDQN)** (không nên nhầm lẫn với DQN kép, mặc dù cả hai kỹ thuật này có thể dễ dàng kết hợp) được giới thiệu trong một bài báo khác vào năm 2015 bởi các nhà nghiên cứu của DeepMind. Để hiểu cách nó hoạt động, trước tiên chúng ta phải lưu ý rằng giá trị Q của một cặp trạng thái-hành động (s, a) có thể được biểu thị là $Q(s, a) = V(s) + A(s, a)$, trong đó $V(s)$ là giá trị của trạng thái s và $A(s, a)$ là **lợi thế (advantage)** của việc thực hiện hành động a ở trạng thái s , so với tất cả các hành động có thể khác trong trạng thái đó. Hơn nữa, giá trị của một trạng thái bằng giá trị Q của hành động tốt nhất a^* cho trạng thái đó (vì chúng ta giả định chính sách tối ưu sẽ chọn hành động tốt nhất), do đó $V(s) = Q(s, a^*)$, điều này ngụ ý rằng $A(s, a^*) = 0$. Trong một DQN đấu, mô hình ước tính cả giá trị của trạng thái và lợi thế của mỗi hành động có thể. Vì hành động tốt nhất nên có lợi thế bằng 0, mô hình trừ đi giá trị lợi thế dự đoán tối đa từ tất cả các lợi thế dự đoán. Đây là một mô hình DDQN đơn giản, được triển khai bằng API chức năng:

```
input_states = tf.keras.layers.Input(shape=[4])
hidden1 = tf.keras.layers.Dense(32, activation="elu")(input_states)
hidden2 = tf.keras.layers.Dense(32, activation="elu")(hidden1)
state_values = tf.keras.layers.Dense(1)(hidden2)
raw_advantages = tf.keras.layers.Dense(n_outputs)(hidden2)
advantages = raw_advantages - tf.reduce_max(raw_advantages, axis=1, keepdims=True)
Q_values = state_values + advantages
model = tf.keras.Model(inputs=[input_states], outputs=[Q_values])
```


Phần còn lại của thuật toán giống như trước đây. Trên thực tế, bạn có thể xây dựng một DQN kép đầu và kết hợp nó với phát lại kinh nghiệm ưu tiên! Tổng quát hơn, nhiều kỹ thuật RL có thể được kết hợp, như DeepMind đã chứng minh trong một bài báo năm 2017: các tác giả của bài báo đã kết hợp sáu kỹ thuật khác nhau thành một tác nhân có tên là Rainbow, vượt trội hơn rất nhiều so với hiệu suất tốt nhất hiện có.

Như bạn có thể thấy, học tăng cường sâu là một lĩnh vực phát triển nhanh chóng và còn nhiều điều để khám phá!

Tổng quan về một số thuật toán RL phổ biến

Trước khi chúng ta kết thúc chương này, hãy xem xét nhanh một vài thuật toán phổ biến khác:

AlphaGo

AlphaGo sử dụng một biến thể của tìm kiếm cây Monte Carlo (MCTS) dựa trên mạng nơ-ron sâu để đánh bại các nhà vô địch cờ vây. MCTS được Nicholas Metropolis và Stanislaw Ulam phát minh vào năm 1949. Nó chọn nước đi tốt nhất sau khi chạy nhiều mô phỏng, liên tục khám phá cây tìm kiếm bắt đầu từ vị trí hiện tại và dành nhiều thời gian hơn cho các nhánh hứa hẹn nhất. Khi nó đạt đến một nút mà nó chưa từng truy cập trước đây, nó chơi ngẫu nhiên cho đến khi trò chơi kết thúc, và cập nhật các ước tính cho mỗi nút đã truy cập (trừ các nước đi ngẫu nhiên), tăng hoặc giảm mỗi ước tính tùy thuộc vào kết quả cuối cùng. AlphaGo dựa trên cùng một nguyên tắc, nhưng nó sử dụng mạng chính sách để chọn nước đi, thay vì chơi ngẫu nhiên. Mạng chính sách này được huấn luyện bằng cách sử dụng độ dốc chính sách. Thuật toán ban đầu bao gồm ba mạng nơ-ron nữa, và phức tạp hơn, nhưng nó đã được đơn giản hóa trong bài báo AlphaGo Zero, sử dụng một mạng nơ-ron duy nhất để cả chọn nước đi và đánh giá trạng thái trò chơi. Bài báo AlphaZero đã khái quát hóa thuật toán này, làm cho nó có khả năng giải quyết không chỉ trò chơi cờ vây, mà còn cờ vua và shogi (cờ tướng Nhật Bản). Cuối cùng, bài báo MuZero tiếp tục cải thiện thuật toán này, vượt trội hơn các lần lặp trước đó mặc dù tác nhân bắt đầu mà thậm chí không biết luật chơi!

Các thuật toán Actor-critic Actor-critics là một nhóm các thuật toán RL kết hợp độ dốc chính sách với mạng Q sâu. Một tác nhân actor-critic chứa hai mạng nơ-ron: một mạng chính sách và một DQN. DQN được huấn luyện bình thường, bằng cách học từ kinh nghiệm của tác nhân. Mạng chính sách học khác (và nhanh hơn nhiều) so với PG thông thường: thay vì ước tính giá trị của mỗi hành động bằng cách trải qua nhiều tập, sau đó tính tổng các phần thưởng tương lai được chiết khấu cho mỗi hành động, và cuối cùng chuẩn hóa chúng, tác nhân (actor) dựa vào các giá trị hành động được ước tính bởi DQN (critic). Điều này hơi giống một vận động viên (tác nhân) học với sự giúp đỡ của một huấn luyện viên (DQN).

Actor-critic lợi thế không đồng bộ (Asynchronous advantage actor-critic - A3C)

Đây là một biến thể actor-critic quan trọng được các nhà nghiên cứu DeepMind giới thiệu vào năm 2016, trong đó nhiều tác nhân học song song, khám phá các bản sao khác nhau của môi trường. Theo các khoảng thời gian đều đặn, nhưng không đồng bộ (do đó có tên), mỗi tác nhân đẩy một số cập nhật trọng số lên mạng chính, sau đó nó kéo các trọng số mới nhất từ mạng đó. Mỗi tác nhân do đó đóng góp vào việc cải thiện mạng chính và hưởng lợi từ

những gì các tác nhân khác đã học. Hơn nữa, thay vì ước tính các giá trị Q , DQN ước tính lợi thế của mỗi hành động (do đó có chữ A thứ hai trong tên), điều này ổn định quá trình huấn luyện.

Actor-critic lợi thế đồng bộ (Advantage actor-critic - A2C) A2C là một biến thể của thuật toán A3C loại bỏ tính không đồng bộ. Tất cả các cập nhật mô hình đều đồng bộ, vì vậy các cập nhật độ dốc được thực hiện trên các lô lớn hơn, cho phép mô hình tận dụng tốt hơn sức mạnh của GPU.

Actor-critic mềm (Soft actor-critic - SAC)

SAC là một biến thể actor-critic được Tuomas Haarnoja và các nhà nghiên cứu UC Berkeley khác đề xuất vào năm 2018. Nó học không chỉ phần thưởng, mà còn để tối đa hóa entropy của các hành động của nó. Nói cách khác, nó cố gắng không thể đoán trước càng nhiều càng tốt trong khi vẫn nhận được nhiều phần thưởng nhất có thể. Điều này khuyến khích tác nhân khám phá môi trường, điều này tăng tốc quá trình huấn luyện, và làm cho nó ít có khả năng lặp lại cùng một hành động khi DQN tạo ra các ước tính không hoàn hảo. Thuật toán này đã chứng minh hiệu quả mẫu đáng kinh ngạc (ngược lại với tất cả các thuật toán trước đó, học rất chậm).

Tối ưu hóa chính sách cận biên (Proximal policy optimization - PPO)

Thuật toán này của John Schulman và các nhà nghiên cứu OpenAI khác dựa trên A2C, nhưng nó cắt xén hàm tổn thất để tránh các cập nhật trọng số quá lớn (thường dẫn đến sự mất ổn định trong huấn luyện). PPO là một sự đơn giản hóa của thuật toán tối ưu hóa chính sách vùng tin cậy (TRPO) trước đó, cũng của OpenAI. OpenAI đã gây chú ý vào tháng 4 năm 2019 với AI của họ có tên OpenAI Five, dựa trên thuật toán PPO, đã đánh bại các nhà vô địch thế giới trong trò chơi nhiều người chơi Dota 2.

Khám phá dựa trên sự tò mò (Curiosity-based exploration)

Một vấn đề tái diễn trong RL là sự thừa thớt của phần thưởng, điều này làm cho việc học rất chậm và không hiệu quả. Deepak Pathak và các nhà nghiên cứu UC Berkeley khác đã đề xuất một cách thú vị để giải quyết vấn đề này: tại sao không bỏ qua phần thưởng, và chỉ làm cho tác nhân cực kỳ tò mò để khám phá môi trường? Phần thưởng do đó trở thành nội tại đối với tác nhân, chứ không phải đến từ môi trường. Tương tự, việc kích thích sự tò mò ở một đứa trẻ có nhiều khả năng mang lại kết quả tốt hơn so với việc chỉ đơn thuần thưởng cho đứa trẻ vì đạt điểm cao. Điều này hoạt động như thế nào? Tác nhân liên tục cố gắng dự đoán kết quả của các hành động của nó, và nó tìm kiếm những tình huống mà kết quả không khớp với dự đoán của nó. Nói cách khác, nó muốn được ngạc nhiên. Nếu kết quả có thể dự đoán được (nhàm chán), nó sẽ đi nơi khác. Tuy nhiên, nếu kết quả không thể đoán trước nhưng tác nhân nhận thấy rằng nó không kiểm soát được nó, nó cũng sẽ chán sau một thời gian. Chỉ với sự tò mò, các tác giả đã thành công trong việc huấn luyện một tác nhân ở nhiều trò chơi điện tử: mặc dù tác nhân không bị phạt khi thua, trò chơi bắt đầu lại, điều này nhàm chán nên nó học cách tránh nó.

Học mở (Open-ended learning - OEL)

Mục tiêu của OEL là huấn luyện các tác nhân có khả năng học vô tận các nhiệm vụ mới và thú vị, thường được tạo ra một cách thủ tục. Chúng ta chưa đạt đến đó, nhưng đã có một số tiến bộ đáng kinh ngạc trong vài năm qua. Ví dụ, một bài báo năm 2019 của một nhóm các nhà nghiên cứu từ Uber AI đã giới thiệu thuật toán POET, tạo ra nhiều môi trường 2D mô phỏng với các va chạm và lỗ hổng và huấn luyện một tác nhân cho mỗi môi trường: mục tiêu của tác nhân là đi bộ nhanh nhất có thể trong khi tránh các chướng ngại vật. Thuật toán bắt đầu với các môi trường đơn giản, nhưng chúng dần dần trở nên khó hơn theo thời gian: đây được gọi là

học theo chương trình (curriculum learning). Hơn nữa, mặc dù mỗi tác nhân chỉ được huấn luyện trong một môi trường, nó phải thường xuyên cạnh tranh với các tác nhân khác, trên tất cả các môi trường. Trong mỗi môi trường, người chiến thắng được sao chép và thay thế tác nhân đã ở đó trước đó. Bằng cách này, kiến thức được thường xuyên chuyển giao giữa các môi trường, và các tác nhân dễ thích nghi nhất được chọn. Cuối cùng, các tác nhân là những người đi bộ tốt hơn nhiều so với các tác nhân được huấn luyện trên một nhiệm vụ duy nhất, và chúng có thể giải quyết các môi trường khó hơn nhiều. Tất nhiên, nguyên tắc này cũng có thể được áp dụng cho các môi trường và nhiệm vụ khác. Nếu bạn quan tâm đến OEL, hãy chắc chắn xem bài báo Enhanced POET, cũng như bài báo năm 2021 của DeepMind về chủ đề này.

Chúng ta đã đề cập đến nhiều chủ đề trong chương này: độ dốc chính sách, chuỗi Markov, quá trình quyết định Markov, học Q, học Q xấp xỉ, và học Q sâu và các biến thể chính của nó (mục tiêu giá trị Q cố định, DQN kép, DQN đấu, và phát lại kinh nghiệm ưu tiên), và cuối cùng chúng ta đã xem nhanh một vài thuật toán phổ biến khác. Học tăng cường là một lĩnh vực rộng lớn và thú vị, với những ý tưởng và thuật toán mới xuất hiện mỗi ngày, vì vậy tôi hy vọng chương này đã khơi gợi sự tò mò của bạn: có cả một thế giới để khám phá!

Bài tập

1. Bạn sẽ định nghĩa học tăng cường như thế nào? Nó khác gì so với học có giám sát hoặc không giám sát thông thường?
2. Bạn có thể nghĩ ra ba ứng dụng có thể của RL mà không được đề cập trong chương này không? Đối với mỗi ứng dụng, môi trường là gì? Tác nhân là gì? Một số hành động có thể là gì? Phần thưởng là gì?
3. Hệ số chiết khấu là gì? Chính sách tối ưu có thể thay đổi nếu bạn sửa đổi hệ số chiết khấu không?
4. Làm thế nào để bạn đo lường hiệu suất của một tác nhân học tăng cường?
5. Bài toán phân bổ tín nhiệm là gì? Khi nào nó xảy ra? Làm thế nào bạn có thể giảm thiểu nó?
6. Mục đích của việc sử dụng bộ đệm phát lại là gì?
7. Thuật toán RL off-policy là gì?
8. Sử dụng độ dốc chính sách để giải quyết môi trường LunarLander-v2 của OpenAI Gym.

9. Sử dụng DQN kép đầu để huấn luyện một tác nhân có thể đạt được cấp độ siêu nhân trong trò chơi Atari Breakout nổi tiếng (“ALE/Breakout-v5”). Các quan sát là hình ảnh. Để đơn giản hóa nhiệm vụ, bạn nên chuyển đổi chúng sang thang độ xám (tức là tính trung bình trên trục kênh) sau đó cắt và giảm kích thước chúng, để chúng chỉ đủ lớn để chơi, nhưng không hơn. Một hình ảnh riêng lẻ không cho bạn biết quả bóng và các mái chèo đang đi theo hướng nào, vì vậy bạn nên hợp nhất hai hoặc ba hình ảnh liên tiếp để tạo thành mỗi trạng thái. Cuối cùng, DQN nên được cấu tạo chủ yếu từ các lớp tích chập.
10. Nếu bạn có khoảng 100 đô dành, bạn có thể mua một Raspberry Pi 3 cộng với một số thành phần robot giá rẻ, cài đặt TensorFlow trên Pi, và thỏa sức sáng tạo! Để có ví dụ, hãy xem bài đăng thú vị này của Lukas Biewald, hoặc xem GoPiGo hoặc BrickPi. Bắt đầu với các mục tiêu đơn giản, như làm cho robot quay xung quanh để tìm góc sáng nhất (nếu nó có cảm biến ánh sáng) hoặc vật thể gần nhất (nếu nó có cảm biến siêu âm), và di chuyển theo hướng đó. Sau đó, bạn có thể bắt đầu sử dụng học sâu: ví dụ, nếu robot có camera, bạn có thể thử triển khai thuật toán phát hiện đối tượng để nó phát hiện người và di chuyển về phía họ. Bạn cũng có thể thử sử dụng RL để làm cho tác nhân tự học cách sử dụng động cơ để đạt được mục tiêu đó. Chúc bạn vui vẻ! Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại <https://homl.info/colab3>.